

Privacy-Preserving in Cloud Networks: An Efficient, Revocable and Authenticated Encrypted Search Scheme

Yibo Cao^{a,b}, Shiyuan Xu^b, Gang Xu^{c,a,*}, Xiu-Bo Chen^a, Yuling Chen^d,
Siu-Ming Yiu^b

^a*Information Security Center, State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, 100876, China*

^b*School of Computing and Data Science, The University of Hong Kong, Pok Fu Lam, 999077, Hong Kong,*

^c*School of Artificial Intelligence and Computer Science, North China University of Technology, Beijing, 100144, China*

^d*State Key Laboratory of Public Big Data, College of Computer Science and Technology, Guizhou University, Guiyang, 550025, China*

Abstract

With the widespread development of cloud networks, performing searches on encrypted data (without decryption) has become a critical issue. Public key authenticated encryption with keyword search (PAEKS) allows for the retrieval of encrypted data while resisting insider keyword guessing attacks (IKGAs). Most PAEKS schemes do not support access control in multi-receiver models. To address this limitation, attribute-based encryption has been introduced. However, the access privileges for the ciphertext may change, and it is vulnerable to quantum computing attacks, which limit its applicability and compromise security in cloud networks. In this paper, we propose RABAEKS, the first revocable and authenticated attribute-based encrypted search scheme over lattice in cloud networks. Our design allows the cloud server to enforce access control for data receivers during the search process. For practical implementation, we introduce a revocation mechanism for receivers, enabling dynamic access control. We then formalize and analyze the security of our scheme rigorously. Through performance evaluations and comparisons, we demonstrate that the search time, ciphertext size, and trapdoor size of our RABAEKS scheme are independent of the number of

*Corresponding author

Email address: `gx@ncut.edu.cn` (Gang Xu)

keywords. Additionally, the computational overhead for ciphertext generation, trapdoor generation, and the search phase in RABAEKS is, at most, 0.91%, 39.21%, and 80.95% of that of previous approaches, respectively, indicating it is efficient and practical in cloud networks.

Keywords: Cloud networks, Privacy-preserving protocol, Authenticated encrypted search, Revocation, Access control

1. Introduction

With the continuous advancement of network technology, cloud networks have emerged as a new solution for data storage and management, significantly enhancing the search and sharing of large-scale data [1], [2]. Data owners can upload large amounts of data to cloud servers (e.g., Amazon Web Services, IBM Cloud), effectively reducing local storage overheads and enhancing service elasticity [3]. Meanwhile, there is also the issue of data privacy leakage in the cloud [4], [5]. To address this issue, many researchers have adopted the encryption-before-outsourcing approach [6], [7] and digital signature technique [8], [9], [10], which poses an obstacle to sharing cloud data. The public key encryption with authenticated keyword search (PAEKS) scheme has been formalized to ensure data privacy without compromising data availability [11]. It incorporates the secret key of the data owner into the encryption algorithm to resist keyword guessing attacks (KGAs) and insider keyword guessing attacks (IKGAs) [12], [13].

Unfortunately, most PAEKS schemes support only the single data receiver model [14], [15], [16], which is not practical in cloud networks, as the data outsourcing process typically involves multiple data receivers. A straightforward approach is to encrypt keywords one by one using the public keys of each receiver, but this would introduce additional computational overheads. To address this limitation, some researchers have proposed multi-receiver PAEKS schemes [17], [18], [19], with the specific process of encrypted search illustrated in Fig. 1. Among these schemes, the most notable is the key-policy attribute-based authenticated encryption with keyword search (ABAEKS) scheme, which integrates key-policy attribute-based encryption (ABE) into PAEKS [20]. This scheme assigns an attribute to the keyword ciphertext, allowing a receiver to access the specific ciphertext that meets the access policy [21], [22]. In comparison to key-policy ABAEKS, the ciphertext-policy ABAEKS scheme enables the data (or keyword) ciphertext to be accessed

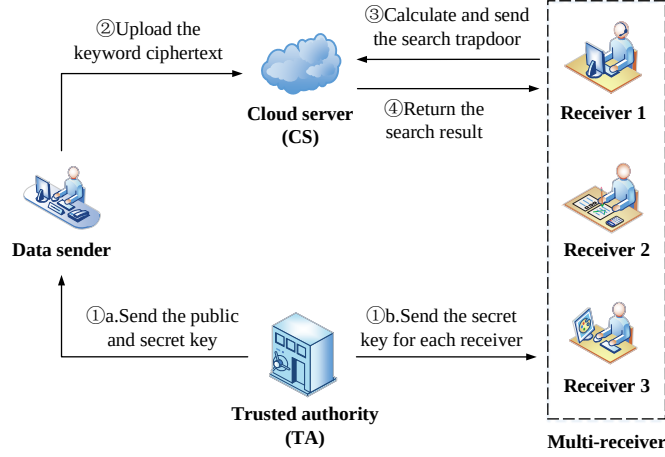


Figure 1: The encrypted search in multi-receiver models.

only by specific data receivers [23], [24], thus facilitating access control in cloud networks. However, the existing ciphertext policy ABAEKS scheme still faces two significant issues in a multi-receiver cloud network.

1.1. Our Motivations and Challenges

On the one hand, in the multi-receiver cloud network, the access privilege for the data (or keyword) ciphertext may dynamically change [25], [26]. For example, consider a medical data platform that stores a large number of patient medical records [27], [28]. Healthcare personnel on the platform are granted access to view specific patient records during designated periods. Nevertheless, if a healthcare worker is reassigned or restricted from accessing certain patients' medical records, their access permissions must be promptly revoked. Such changes in access permissions necessitate the re-encryption of the data ciphertext. To bridge the gap, the revocation mechanism plays a crucial role in enabling dynamic access control, allowing for the immediate revocation of a receiver's access to sensitive data when access permissions change [29], [30], [31], [32]. This mechanism typically requires the cloud server to maintain a revocation list, ensuring that the data (or keyword) ciphertext becomes inaccessible to the receivers on that list. When the revocation list is updated, the data (or keyword) ciphertext is also modified accordingly.

On the other hand, quantum computers present a significant security

threat to conventional cryptography [33], [34]. For instance, Shor’s algorithm can solve the discrete logarithm (DL) problem in probabilistic polynomial time (PPT) [35]. Once large-scale quantum computers become practical, the traditional public-key primitives securing these systems would be rendered vulnerable, thereby exposing outsourced data to unprecedented risks. To counter this threat, lattice-based cryptography has emerged as a promising approach to resist quantum computing attacks [36], [37]. In particular, the learning with errors (LWE) assumption has been commonly used in several lattice-based SE schemes [38], [39], [40], as it is as challenging as the worst-case standard lattice problems.

Therefore, it is crucial to construct a lattice-based revocable ABAEKS scheme for cloud networks, which can not only resist quantum computing attacks but also support dynamic access control, thereby enabling data privacy-preserving in multi-receiver cloud scenarios.

The existing ABAEKS scheme is facing three challenges. Firstly, in [20], the attribute is embedded in the keyword ciphertext (aka, key-policy), which cannot provide attributes to each data receiver. In other words, there exists no ciphertext-policy ABAEKS until now. Secondly, it is not possible to revoke access privileges for a specific data receiver. Thirdly, how to generate the valid keyword ciphertext and search trapdoor, and then match them in the search phase.

1.2. Our Contributions

Addressing the above-mentioned challenges is not trivial. In this paper, we propose RABAEKS, a revocable and authenticated attribute-based encrypted search scheme over lattice in cloud networks, which enables dynamic access control and resists quantum computing attacks and IKGAs.

To begin with, we leverage the philosophy of lattice hardness to achieve post-quantum security. Different from existing schemes, we initialize an attribute set $\mathcal{ATT}$ and many attribute matrices corresponding to each element in it. Then, we construct a receiver attribute set $\mathcal{S}$ embedded in its secret key and an access policy $(\mathcal{W}, t)$ embedded in the keyword ciphertext, and apply the Lagrange interpolation technique to design a ciphertext-policy solution.

With regard to the revocation mechanism, we implement the revocation list $\mathcal{R}$ utilizing the binary tree structure [31] and **KUNodes** algorithm [41], subsequently revoking the access permission of a specific receiver in the search phase.

In the ciphertext generation, the secret key of the data sender is input to authenticate the keyword through the **SampleLeft** algorithm, so a malicious attacker cannot launch IKGAs by generating a valid ciphertext. To generate a valid trapdoor, a straightforward way is to utilize the **SamplePre** or **SampleLeft** algorithm to calculate a trapdoor. However, after embedding the matrices in a binary tree structure into the ciphertext and trapdoor, these two algorithms cannot be invoked by the receiver’s secret key. To address this, we input the attribute matrix into the **SampleBasisLeft** algorithm to calculate a secret key of data receivers, and initially introduce the **GenSamplePre** algorithm by using this secret key to generate a vector as part of the search trapdoor. After a data receiver submits a trapdoor, the cloud server checks whether the receiver’s attributes satisfy the access policy and whether the receiver is not listed in the revocation list. If both conditions are met, the cloud server proceeds to search for the keyword ciphertext corresponding to the submitted trapdoor, thereby enabling the receiver revocation and encrypted search.

We hereby summarize our four-fold contributions as follows.

- We propose a revocable and authenticated attribute-based encrypted search scheme over lattice in cloud networks, namely RABAEKS. This scheme not only supports multi-receiver and dynamic access control but also offers resistance to quantum computing attacks and IKGAs. To the best of our knowledge, our design is the first lattice-based ciphertext-policy ABAEKS scheme with receiver revocation.
- We leverage the lattice algebraic structure to achieve post-quantum security in cloud networks, utilizing several lattice sampling algorithms. From a higher perspective, the keyword ciphertext is authenticated by the sender’s secret key through the **SampleLeft** algorithm. A search trapdoor is then calculated using the **GenSamplePre** algorithm to enable encrypted search.
- We implement the access control structure using the threshold secret sharing technique based on Lagrange interpolation. Additionally, we introduce a revocation list, a binary tree structure, and the **KUNodes** algorithm to facilitate receiver revocation and ciphertext updates, thereby enabling dynamic multi-receiver access control in cloud networks.
- We present the security model and security analysis in the random oracle model (ROM), which demonstrates that RABAEKS satisfies IND-

CKA and IND-IKGA security in a quantum setting. Furthermore, comprehensive performance evaluations show that the search time, ciphertext size, and trapdoor size of RABAEKS are independent of the number of keywords. As an efficient scheme, RABAEKS can process multiple keywords simultaneously, and outperforms prior arts [33], [38], [39], [20] in terms of ciphertext generation, trapdoor generation, and search algorithms. Specifically, the computational overhead for these three algorithms is, at most, 0.91%, 39.21%, and 80.95% of that of prior arts, respectively.

1.3. Organization

The rest of this paper is organized as follows. Section 2 presents numerous related works to showcase recent advancements. Following that, Section 3 provides an introduction to the preliminary concepts. The system models, threat models, formal definitions, and security models for RABAEKS are then depicted in Section 4. A detailed construction of RABAEKS is demonstrated in Section 5, while Section 6 focuses on the security analysis. In Section 7, we delve into the performance evaluation and comparison. Finally, we summarize this paper in Section 8.

2. Related Works

We review the literature in the context of PAEKS, attribute-based encrypted search and revocation schemes, and lattice-based solutions.

2.1. Public Key Authenticated Encryption with Keyword Search

Huang et al. [11] formalized the first PAEKS scheme to resist IKGAs during encrypted search. In their scheme, a data sender not only encrypts the keywords but also authenticates them with its secret key so that the keyword ciphertext can only be calculated by itself. Following this, a great deal of PAEKS schemes have been conducted. For example, Li et al. [12] proposed a PAEKS scheme with ciphertext update, ensuring the security of electronic medical records (EMRs) in hospital information systems. Cheng et al. [13] designed a server-aided PAEKS scheme and introduced a ciphertext deduplication method to improve the search efficiency. Wu et al. [14] constructed a PEKS scheme based on a cryptographic reverse firewall, which can resist IKGAs and algorithm substitution attacks. To tackle the issue of poor flexibility in the encrypted search, Luo et al. [16] presented a PAEKS

scheme with wildcard search. This scheme has a constant complexity with the increasing number of wildcards. However, those schemes still suffer from poor utility in cloud networks.

In order to improve the practicality, numerous researchers contributed to designing the multi-receiver PAEKS schemes [17], [18], [19]. Chenam et al. [17] constructed a designated cloud server-based multi-receiver certificateless PAEKS with conjunctive search. After that, Sun et al. [18] presented a multi-receiver certificateless PAEKS for concealing search patterns, without the need for expensive bilinear pairing operations. Recently, a new lightweight multi-receiver PAEKS scheme was proposed to [19]. This scheme implements a specialized server that effectively simplifies the many-to-many connections between senders and receivers into a one-to-one relationship. Nevertheless, none of these aforementioned schemes can support access control.

2.2. Attribute-based Encrypted Search and Revocation Schemes

Attribute-based encryption with keyword search (ABKS) schemes can provide an encrypted search under a multi-receiver scenario to achieve data privacy-preserving for cloud networks. For instance, Huang et al. [23] utilized a Bloom filter tree structure to propose an ABKS system for cloud networks. After that, Yin et al. [24] presented a new decentralized ciphertext-policy ABKS scheme, which not only ensures the security of the cloud data but also achieves the access control in a fine-grained manner. Most recently, Ghopur et al. [25] designed a decentralized multi-authority ABKS primitive to realize access control by leveraging the philosophy of ABE. Zhang et al. [28] constructed an efficient ABKS scheme with policy hiding in healthcare services. In this scheme, the access policy is concealed to protect sensitive information from potential exposure. However, when the data receiver's status was changed, they could not offer dynamic access control.

Revocation mechanism can dynamically adjust the access privilege of data receivers when their status has been changed, which serves as a cornerstone for the access control systems [29]. To achieve the revocation, Miao et al. [42] proposed a time-controllable keyword search scheme through introducing the ABE, and provided an efficient receiver revocation through the secret sharing technique. In addition, Meng et al. [43] presented a server-aided traceable and revocable ABKS scheme. In their scheme, revoked receivers cannot access either the newly uploaded files or the old ones, thereby protecting data privacy. Along with this direction, Hou et al. [26] formulated an ABKS system based on blockchain, named BC-RMABSE, which further considers the

ranked keyword search and policy hiding. However, most existing schemes are constructed based on the DL hardness and are not resilient to quantum computing attacks.

2.3. Lattice-based PAEKS and Revocation Schemes

With the development of quantum computers, conventional cryptographic primitives have been seriously threatened. Lattice-based cryptography is considered the most promising post-quantum solution, which has been widely utilized in both PAEKS and revocation schemes.

In 2022, Liu et al. [44] proposed a generic construction of PAEKS and a lattice-based instantiation. Then, Cheng et al. [33] enhanced the security level of the lattice-based PAEKS scheme to the fully ciphertext indistinguishability (fully CI) and fully trapdoor indistinguishability (fully TI). To support the multi-receiver model, Luo et al. formalized a key-policy ABAEKS scheme [20] and authenticated re-encryption with keyword search (Re-PAEKS) [45] over lattice, to realize the IND-CKA and IND-IKGA security under quantum computing attacks.

With regard to the revocation mechanism, a revocable ABE scheme over lattice was presented by Zhao et al. [46] for privacy-preserving in cloud networks, and its security was reduced to the Ring-LWE hardness. After that, Luo et al. [30] constructed a revocable key policy ABE scheme reduced on LWE hardness, which utilizes the **KUNodes** algorithm to realize revocation. Nevertheless, during the revocation process of their scheme, the secret key of the data receiver is necessary, which causes secret key leakage concerns. To mitigate this, Guo et al. [31] proposed a novel lattice-based revocable ciphertext policy ABE scheme that also introduces the **KUNodes** algorithm. Along this direction, [32] put forward a multi-authority ciphertext-policy ABKS with attribute revocation in the cloud.

Consequently, to the best of our knowledge, none of the existing ABAEKS schemes can support receiver revocation, but also resist quantum computing attacks.

3. Preliminaries

In this section, we introduce the preliminaries toward our RABAEKS scheme, i.e., lattice definition, discrete Gaussian distribution, LWE hardness, and lattice basis sampling algorithms.

Definition 1. [47] Assume that a matrix $\mathbf{E} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_m)$ is composed of m linearly independent vectors, we define the lattice Λ as:

$$\Lambda = \Lambda(\mathbf{E}) = \{x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + \dots + x_m\mathbf{e}_m \mid x_i \in \mathbb{Z}, i \in [m]\}.$$

We say that $\mathbf{E}$ is a lattice basis of Λ .

Definition 2. [48] Given three integers n, m, q , and a matrix $\mathbf{E} \in \mathbb{Z}_q^{n \times m}$, we define a q -ary integer lattice as:

$$\Lambda_q(\mathbf{E}) := \{\mathbf{h} \in \mathbb{Z}^m \mid \exists \mathbf{s} \in \mathbb{Z}_q^n, \mathbf{E}^\top \mathbf{s} = \mathbf{h} \bmod q\}.$$

$$\Lambda_q^\perp(\mathbf{E}) := \{\mathbf{h} \in \mathbb{Z}^m \mid \mathbf{E}\mathbf{h} = \mathbf{0} \bmod q\}.$$

$$\Lambda_q^{\mathbf{u}}(\mathbf{E}) := \{\mathbf{h} \in \mathbb{Z}^m \mid \mathbf{E}\mathbf{h} = \mathbf{u} \bmod q\}.$$

Definition 3. Given a parameter $\sigma \in \mathbb{R}^+$, a center $\mathbf{c} \in \mathbb{Z}^m$, and any vector $\mathbf{x} \in \mathbb{Z}^m$, we define the discrete Gaussian distribution over Λ as: $\mathcal{D}_{\Lambda, \sigma, \mathbf{c}}(\mathbf{x}) = \frac{\rho_{\sigma, \mathbf{c}}(\mathbf{x})}{\rho_{\sigma, \mathbf{c}}(\Lambda)}$. For $\forall \mathbf{x} \in \Lambda$, we say that $\rho_{\sigma, \mathbf{c}}(\mathbf{x}) = \exp(-\pi \frac{\|\mathbf{x} - \mathbf{c}\|^2}{\sigma^2})$, and $\rho_{\sigma, \mathbf{c}}(\Lambda) = \sum_{\mathbf{x} \in \Lambda} \rho_{\sigma, \mathbf{c}}(\mathbf{x})$.

Definition 4. [49] Given three integers n, m, q , and an error distribution Ψ_α , we define the $\text{LWE}_{n, m, q, \Psi_\alpha}$ hardness as distinguishing a pseudo-random tuple $(\mathbf{A}, \mathbf{A}^\top \mathbf{s} + \mathbf{e})$ and a random tuple $(\mathbf{A}, \mathbf{x})$, where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$, $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, $\mathbf{e} \leftarrow \Psi_\alpha^m$, and $\mathbf{x} \leftarrow \mathbb{Z}_q^m$.

Lemma 1. [50] Given a lattice Λ and its lattice basis $\mathbf{T}_\mathbf{A}$, we can say: $\Pr[\|\mathbf{x}\| > \sigma\sqrt{m} : \mathbf{x} \leftarrow \mathcal{D}_{\Lambda, \sigma}] \leq \text{negl}(m)$, and $\sigma \geq \|\widetilde{\mathbf{T}}_\mathbf{A}\| \cdot \omega(\sqrt{\log m})$.

Lemma 2. [51] Given three integers $n, q \geq 2$, and $m \geq 2n \log q$, the probabilistic polynomial time (PPT) algorithm $\text{TrapGen}(n, m, q)$ computes a uniform matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and its basis $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m}$ over $\Lambda_q^\perp(\mathbf{A})$. We can say that $\mathbf{A}$ is statistically close to uniform distribution over $\mathbb{Z}^{n \times m}$ and $\|\widetilde{\mathbf{T}}_\mathbf{A}\| \leq \mathcal{O}(\sqrt{n \log m})$.

Lemma 3. [52] Taking four integers q, n, m, k , two matrices $\mathbf{E}_1 \in \mathbb{Z}_q^{n \times m}$, $\mathbf{E}_2 \in \mathbb{Z}_q^{n \times k}$, a lattice basis $\mathbf{T}_{\mathbf{E}_1}$ for $\Lambda_q^\perp(\mathbf{E}_1)$, and a parameter $\sigma > \|\widetilde{\mathbf{T}}_{\mathbf{E}_1}\| \omega(\sqrt{\log r})$ as inputs, the $\text{SampleBasisLeft}(\mathbf{E}_1, \mathbf{E}_2, \mathbf{T}_{\mathbf{E}_1}, \sigma)$ algorithm computes a lattice basis $\mathbf{T}_\mathbf{E}$ for $\Lambda_q^\perp(\mathbf{E})$, where the matrix $\mathbf{E} = (\mathbf{E}_1 \mid \mathbf{E}_2)$.

Lemma 4. [52] Taking four integers q, n, m, k , three matrices $\mathbf{E}_1 \in \mathbb{Z}_q^{n \times m}$, $\mathbf{E}_2 \in \mathbb{Z}_q^{n \times k}$, $\mathbf{R} \in \mathbb{Z}_q^{m \times k}$, a lattice basis $\mathbf{T}_{\mathbf{E}_2}$ for $\Lambda_q^\perp(\mathbf{E}_2)$, and a parameter $\sigma > \|\widetilde{\mathbf{T}_{\mathbf{E}_2}}\| \cdot \|\mathbf{R}\| \cdot \omega(\sqrt{\log m})$ as inputs, the $\text{SampleBasisRight}(\mathbf{E}_1, \mathbf{E}_2, \mathbf{R}, \mathbf{T}_{\mathbf{E}_2}, \sigma)$ algorithm computes a lattice basis $\mathbf{T}_{\mathbf{E}}$ for $\Lambda_q^\perp(\mathbf{E})$, $\mathbf{E} = (\mathbf{E}_1 \mid \mathbf{E}_1\mathbf{R} + \mathbf{E}_2)$.

Lemma 5. [52] Taking a matrix $\mathbf{E} \in \mathbb{Z}_q^{n \times m}$ and its basis $\mathbf{T}_{\mathbf{E}} \in \mathbb{Z}_q^{m \times m}$, a vector $\mathbf{t} \in \mathbb{Z}_q^n$, a matrix $\mathbf{H} \in \mathbb{Z}_q^{n \times k}$, and a parameter $\sigma \leq \|\widetilde{\mathbf{T}_{\mathbf{E}}}\| \cdot \omega(\sqrt{\log(m+k)})$ as inputs, the $\text{SampleLeft}(\mathbf{E}, \mathbf{H}, \mathbf{T}_{\mathbf{E}}, \mathbf{t}, \sigma)$ algorithm computes a vector $\mathbf{s} \in \mathbb{Z}^{m+k}$, which is statistically close over the distribution $\mathcal{D}_{\Lambda_q^\perp([\mathbf{E}|\mathbf{H}]}, \sigma$, satisfying $[\mathbf{E}|\mathbf{H}] \cdot \mathbf{s} = \mathbf{t} \pmod{q}$.

Lemma 6. [52] Taking two matrices $\mathbf{E}, \mathbf{M} \in \mathbb{Z}_q^{n \times m}$ and a basis $\mathbf{T}_{\mathbf{M}}$ over $\Lambda_q^\perp(\mathbf{M})$, a matrix $\mathbf{H} \in \mathbb{Z}_q^{m \times m}$, a vector $\mathbf{v} \in \mathbb{Z}_q^n$, and $\sigma > \|\widetilde{\mathbf{T}_{\mathbf{M}}}\| \cdot s_1(\mathbf{H}) \cdot \omega(\sqrt{\log m})$ as inputs, the $\text{SampleRight}(\mathbf{E}, \mathbf{M}, \mathbf{H}, \mathbf{T}_{\mathbf{M}}, \mathbf{v}, \sigma)$ algorithm computes a vector $\mathbf{t} \in \mathbb{Z}^{2m}$, which is statistically close to the distribution over $\mathcal{D}_{\Lambda_q^\perp([\mathbf{E}|\mathbf{E}\mathbf{H}+\mathbf{M}]}, \sigma$.

Assume that four positive integers $n, q \geq 2, \tau, m \geq 2n \log q$, a matrix $\mathbf{E} = (\mathbf{E}_1 \mid \mathbf{E}_2 \mid \cdots \mid \mathbf{E}_\tau) \in \mathbb{Z}_q^{n \times \tau m}$, and a set $\mathcal{N} = \{i_1, i_2, \dots, i_j\} \subset [\tau]$, we let $\mathbf{E}_{\mathcal{N}} := (\mathbf{E}_{i_1} \mid \mathbf{E}_{i_2} \mid \cdots \mid \mathbf{E}_{i_j}) \in \mathbb{Z}_q^{n \times jm}$. Now, we provide the following **Lemma 7.**

Lemma 7. [53] Taking a matrix $\mathbf{E} \in \mathbb{Z}_q^{n \times \tau m}$, a lattice basis $\mathbf{T}_{\mathbf{E}_{\mathcal{N}}}$ over $\Lambda_q^\perp(\mathbf{E}_{\mathcal{N}})$, a set $\mathcal{N} \subset [\tau]$, a vector $\mathbf{u} \in \mathbb{Z}_q^n$, and a parameter $\sigma \geq \|\widetilde{\mathbf{T}_{\mathbf{E}_{\mathcal{N}}}}\| \cdot \omega(\sqrt{\log \tau m})$ as inputs, the $\text{GenSamplePre}(\mathbf{E}, \mathbf{T}_{\mathbf{E}_{\mathcal{N}}}, \mathcal{N}, \mathbf{u}, \sigma)$ algorithm computes a vector $\mathbf{e} \in \mathbb{Z}^{\tau m}$, which is statistically close to the distribution over $\mathcal{D}_{\Lambda_q^\perp(\mathbf{E}), \sigma}$, satisfying $\mathbf{E}\mathbf{e} = \mathbf{u} \pmod{q}$.

Definition 5. [31], [41] Assuming that a set of data receivers is $\mathcal{U} := \{1, \dots, |\mathcal{U}|\}$, a revocation list is $\mathcal{R}$, and a root matrix is $\mathbf{B}_0 \in \mathbb{Z}_q^{n \times m}$, we construct a binary tree $\mathcal{T}$ structure as showed in Fig. 2. For the non-leaf nodes $i \in [1, |\mathcal{U}| - 2]$ and the leaf nodes $j \in [|\mathcal{U}| - 1, 2|\mathcal{U}| - 2]$, we calculate $\mathbf{B}_i = \mathbf{B}_0 \mathbf{U}_i$, and $\mathbf{B}_j = \mathbf{B}_0 \mathbf{V}_j$, where $\mathbf{U}_i \leftarrow \{0, 1\}^{m \times m}$ is a reversible matrix such that each row and column has only one unit element and the rest of elements are zero, $\mathbf{V}_j \leftarrow \{-1, 1\}^{m \times m}$ is a reversible matrix. In $\mathcal{T}$, each leaf node corresponds to a data receiver in $\mathcal{U}$, and $\omega(\xi)$ represents the iteration of each node number in the path from root to this leaf. For example, $\omega(3) = \{0, 1, 4, 9\}$, and $\omega(5) = \{0, 2, 5, 11\}$. Then, we

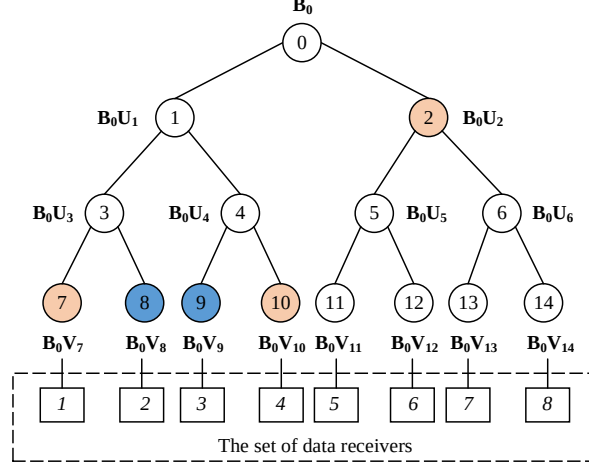


Figure 2: The binary tree $\mathcal{T}$ for revocation.

define the $\text{KUNodes}(\mathcal{T}, \mathcal{R})$ as Algorithm 1. For instance, if $\mathcal{R} = \{2, 3\}$, the $\text{KUNodes}(\mathcal{T}, \mathcal{R}) = \{2, 7, 10\}$ according to **Algorithm 1**. If a data receiver (i.e. leaf node) ξ in the revocation list $\mathcal{R}$, $\text{KUNodes}(\mathcal{T}, \mathcal{R}) \cap \omega(\xi) = \emptyset$. Otherwise, $\text{KUNodes}(\mathcal{T}, \mathcal{R}) \cap \omega(\xi)$ has one and only one element.

Definition 6. Given a hash function family $\mathcal{H} := \{H : X \rightarrow Y\}$ where $0 \in Y$ and a $(Q + 1)$ -dimension vector $\mathbf{x} = (x_0, \dots, x_Q) \in X^{Q+1}$ as inputs, we define that $\mathcal{H}$ is $(Q, p_{\min}, p_{\max})$ abort-resistant if $p(\mathbf{x}) := \Pr[H(x_0) = 0 \cap H(x_1) \neq 0 \cap \dots \cap H(x_Q) \neq 0] \in [p_{\min}, p_{\max}]$ for all $\mathbf{x} \in X^{Q+1}$ with $x_0 \neq \{x_1, \dots, x_Q\}$ and $H \in \mathcal{H}$. Specifically, we define a abort-resistant hash function $H : \mathbb{Z}_q^k \setminus \{0\}^k \rightarrow \mathbb{Z}_q$ as $H(\mathbf{m}) := 1 + \sum_{i=1}^k \theta_i m_i$, where q is a prime, $\boldsymbol{\theta} := (\theta_1, \dots, \theta_k) \in \mathbb{Z}_q^k$, and $\mathbf{m} := (m_1, \dots, m_k) \in \mathbb{Z}_q^k \setminus \{0\}^k$.

Lemma 8. [52] Assume that q is a prime and $0 \leq Q \leq q$, the hash function $H : \mathbb{Z}_q^k \setminus \{0\}^k \rightarrow \mathbb{Z}_q$ defined in **Definition 6** is $(Q, \frac{1}{q}(1 - \frac{Q}{q}), \frac{1}{q})$ abort-resistant.

4. Framework Formulation

In this section, we show the system models, threat models, formal definitions, and security models. The notations utilized in our scheme are defined in Table 1.

Algorithm 1 KUNodes

Input: A binary tree $\mathcal{T}$, and a revocation list $\mathcal{R}$.

Output: A set $\mathcal{Y}$.

```
1 Initialize two sets  $\mathcal{X} \leftarrow \emptyset, \mathcal{Y} \leftarrow \emptyset$ .
2 for  $i \in \mathcal{R}$  do
3   | Calculate  $\mathcal{X} = \mathcal{X} \cup \omega(i)$ .
4 end
5 for  $j \in \mathcal{X}$  do
6   | if  $j_{\text{LeftNode}} \notin \mathcal{X}$  then
7     | Calculate  $\mathcal{Y} = \mathcal{Y} \cup \{j_{\text{LeftNode}}\}$ .
8   | end
9   | if  $j_{\text{RightNode}} \notin \mathcal{X}$  then
10    | Calculate  $\mathcal{Y} = \mathcal{Y} \cup \{j_{\text{RightNode}}\}$ .
11   | end
12 end
13 if  $\mathcal{Y} = \emptyset$  then
14   | Set  $\mathcal{Y} = \mathcal{Y} \cup \{0\}$ .
15 end
16 return A set  $\mathcal{Y}$ .
```

4.1. System Models

In our RABAEKS scheme, there are four entities involved: Trusted authority (TA), Data sender, Data receivers, and Cloud server (CS), as depicted in Fig. 3.

The TA is in charge of initializing the system and generating the key for the data sender and receiver. For a data sender, the TA calculates its public and secret keys according to the public parameter. For a data receiver, the TA calculates its secret key according to the public parameter, its attribute, and the master secret key.

The data sender has a collection of data files and extracts the keyword from them. After receiving the public and secret key from the TA, the data sender encrypts the keyword with its secret key and an access policy through invoking the **Encrypt** algorithm, to obtain the keyword ciphertext that can only be accessed by some specific data receivers. Finally, the data sender uploads the ciphertext to CS.

Each data receiver owns an attribute in our scheme, which can be utilized to generate its secret key in the TA. When the data receiver has a search

Table 1: Nomenclature

Symbol	Description
λ	The security parameter
$ kw $	The length of keyword
$ att $	The number of system attribute
l	The number of receiver attribute, where $l \leq att $
$\mathcal{ATT}$	The system attribute, where $\mathcal{ATT} := \{1, \dots, att \}$
$\mathcal{U}$	The set of data receivers
$\mathcal{R}$	The revocation list
pp	The public parameter
msk	The system master secret key
(pk_S, sk_S)	The public and secret key of data sender
$\mathcal{S}$	The receiver attribute, where $\mathcal{S} \subseteq \mathcal{ATT}$
sk_R	The secret key of data receiver
$(\mathcal{W}, t)$	The access policy, where $\mathcal{W} \subset \mathcal{ATT}$, and $t \in [1, att)$
$\mathbf{ck}$	The ciphertext keyword, where $\mathbf{ck} \in \{0, 1\}^{ kw }$
CT	The keyword ciphertext
$\mathbf{tk}$	The trapdoor keyword, where $\mathbf{tk} \in \{0, 1\}^{ kw }$
TD	The search trapdoor

requirement, it calculates a search trapdoor using the **Trapdoor** algorithm and sends it to CS. If the attribute meets the access condition (i.e., meets the attribute policy $(\mathcal{W}, t)$, and is not in the revocation list) and the trapdoor matches the corresponding ciphertext, the data receiver obtains the search result from the CS.

The CS provides the data storage and access control functions. It stores the keyword ciphertext with an access policy and a revocation list (RL). After receiving a search trapdoor from a data receiver, CS checks whether its attribute meets the access condition, including whether its attribute meets the access policy defined by the data sender, and it is not in the RL. If the access condition is matched, CS performs the **Search** algorithm and sends the corresponding search result to the data receiver. Moreover, when the RL is updated, CS invokes the **CTUpdate** algorithm to refresh the keyword ciphertext.

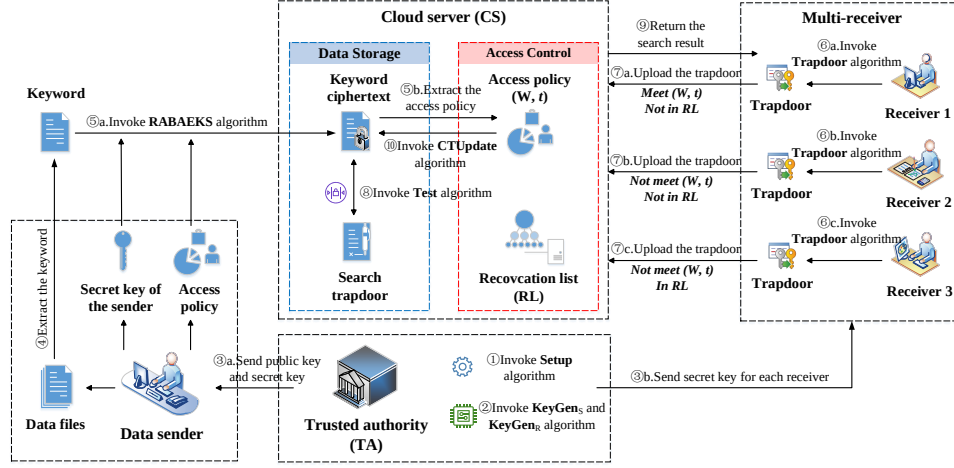


Figure 3: System model of our RABAEKS for cloud networks.

4.2. Threat Models

Assuming that the communication channel is secure, we present the threat assumptions of four entities as follows.

- **TA** is designed to be *fully trusted*. It generates valid public and secret keys for data sender and receivers, and is not interested in the data files and keywords stored in the cloud server.
- **Data Sender** is considered to be *fully trusted*. It protects the secret key and executes the encryption algorithm honestly to generate valid keyword ciphertexts for the cloud server.
- **Data Receivers** are assumed to be *malicious*. They generate valid trapdoors according to the keyword to be searched, while they are interested in speculating on the search context of other receivers by performing an unauthorised search.
- **CS** is considered to be *honest-but-curious*. It strictly executes the search algorithm and returns the correct search results to data receivers that satisfy the access conditions, while it is eager to obtain the data files and keywords.

4.3. Formal Definitions

Our RABAEKS scheme consists of seven algorithms $\Pi_{\text{RABAEKS}} = (\text{Setup}, \text{KeyGen}_S, \text{KeyGen}_R, \text{Encrypt}, \text{Trapdoor}, \text{Search}, \text{CTUpdate})$. We describe their formal definitions as follows.

- $(pp, msk) \leftarrow \text{Setup}(\lambda)$: Given a security parameter λ , this algorithm returns a public parameter pp and a master secret key msk .
- $(pk_S, sk_S) \leftarrow \text{KeyGen}_S(pp)$: Given a public parameter pp , this algorithm returns the public and secret keys (pk_S, sk_S) to the data sender.
- $sk_R \leftarrow \text{KeyGen}_R(pp, \mathcal{S}, msk)$: Given a public parameter pp , a receiver attribute set $\mathcal{S} \subseteq \mathcal{ATT}$, and a master secret key msk , this algorithm returns the secret keys sk_R to the data receiver.
- $\text{CT} \leftarrow \text{Encrypt}(pp, pk_S, sk_S, (\mathcal{W}, t), \mathbf{ck})$: Given a public parameter pp , a data sender's public and secret keys pk_S and sk_S , an access policy $(\mathcal{W}, t)$, and a ciphertext keyword $\mathbf{ck}$, this algorithm returns the keyword ciphertext CT to the cloud server.
- $\text{TD} \leftarrow \text{Trapdoor}(pp, pk_S, sk_R, (\mathcal{S}, \xi), \mathbf{tk})$: Given a public parameter pp , a data sender's public key pk_S , a data receiver's secret key sk_R , an attribute set $\mathcal{S}$ corresponding to the data receiver ξ , and a trapdoor keyword $\mathbf{tk}$, this algorithm returns the trapdoor TD to the cloud server.
- $1 \text{ or } 0 \leftarrow \text{Search}(pp, (\mathcal{S}, \xi), (\mathcal{W}, t), \text{CT}, \text{TD})$: Given a public parameter pp , an attribute set $\mathcal{S}$ corresponding to the data receiver ξ , an access policy $(\mathcal{W}, t)$, a keyword ciphertext CT, and a trapdoor TD, this algorithm returns 1 or 0 to represent the ciphertext and trapdoor whether correspond to the same keyword.
- $\text{CT}' \leftarrow \text{CTUpdate}(pp, \mathcal{R}', \text{CT})$: Given a public parameter pp , an updated revocation list $\mathcal{R}'$, and a keyword ciphertext CT, this algorithm returns an updated keyword ciphertext CT' to the cloud server.

4.4. Security Models

4.4.1. IND-CKA security

We define the IND-CKA security model $\text{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda)$ as below.

1. **Setup:** The adversary $\mathcal{A}$ sends a challenge access structure $(\mathcal{W}^*, t^*)$ and a revocation list $\mathcal{R}^*$ to the challenger $\mathcal{C}$, where $t^* \in [1, |att|]$. $\mathcal{C}$ invokes the **Setup** (λ) algorithm to calculate a public parameter pp and a master secret key msk .
2. **Phase 1:** In polynomial times, $\mathcal{A}$ perform the following queries in an adaptive manner.
 - (a) **Secret Key Queries** $\mathcal{O}_{RK}$: $\mathcal{A}$ submits (pp, S, msk) to $\mathcal{C}$, and then $\mathcal{C}$ invokes **KeyGen** $_R$ algorithm to compute a secret key sk_R . Ultimately, $\mathcal{C}$ returns sk_R to $\mathcal{A}$.
 - (b) **Ciphertext Queries** $\mathcal{O}_{CT}$: $\mathcal{A}$ delivers $((\mathcal{W}, t), \mathbf{ck})$ to $\mathcal{C}$, and then $\mathcal{C}$ executes **Encrypt** $(pp, pk_S, sk_S, (\mathcal{W}, t), \mathbf{ck})$ algorithm to compute a ciphertext CT associated to $\mathbf{ck}$. Eventually, $\mathcal{C}$ returns CT to $\mathcal{A}$.
 - (c) **Trapdoor Queries** $\mathcal{O}_{TD}$: $\mathcal{A}$ sends $((\mathcal{S}, \xi), \mathbf{tk})$ to $\mathcal{C}$, and then $\mathcal{C}$ executes **Trapdoor** $(pp, pk_S, sk_R, (\mathcal{S}, \xi), \mathbf{tk})$ algorithm to compute a trapdoor TD associated to $\mathbf{tk}$. Finally, $\mathcal{C}$ returns TD to $\mathcal{A}$.
3. **Challenge:** $\mathcal{A}$ chooses $\mathbf{ck}_0^*, \mathbf{ck}_1^*$ which have not been queried in **Phase 1** as two challenge ciphertext keywords, and sends them to $\mathcal{C}$. After that, $\mathcal{C}$ selects a random bit $\mu \in \{0, 1\}$ and invokes the **Encrypt** $(pp, pk_S, sk_S, (\mathcal{W}^*, t^*), \mathbf{ck}_\mu^*)$ algorithm to obtain a challenge ciphertext CT_μ^* . Ultimately, $\mathcal{C}$ returns CT_μ^* to $\mathcal{A}$.
4. **Phase 2:** $\mathcal{A}$ continues to access $\mathcal{O}_{RK}$, $\mathcal{O}_{CT}$, and $\mathcal{O}_{TD}$ oracles, but subject to the restriction with neither $\mathbf{ck}_0^*$ nor $\mathbf{ck}_1^*$ can be queried.
5. **Guess:** $\mathcal{A}$ guesses a bit $\mu' \in \{0, 1\}$. If $\mu' = \mu$, we say that $\mathcal{A}$ wins this game.

We hereby define the advantage of $\mathcal{A}$ to win the above game $\mathbf{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda)$ as:

$$\text{Adv}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda) = |\Pr[\mu' = \mu] - \frac{1}{2}|.$$

Definition 7. *Our RABAEKS primitive satisfies IND-CKA security, if any PPT adversary wins the above game $\mathbf{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda)$ with a negligible advantage.*

4.4.2. IND-IKGA security

We define the IND-IKGA security model $\mathbf{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-IKGA}}(\lambda)$ as below.

1. **Setup**: The adversary $\mathcal{A}$ sends a challenge attribute $(\mathcal{S}^*, \xi^*)$ and a revocation list $\mathcal{R}^*$ to the challenger $\mathcal{C}$, where $t^* \in [1, |att|]$. $\mathcal{C}$ invokes the **Setup** (λ) algorithm to calculate a public parameter pp and a master secret key msk .
2. **Phase 1**: In polynomial times, $\mathcal{A}$ perform the following queries in an adaptive manner.
 - (a) **Secret Key Queries** $\mathcal{O}_{RK}$: The query is same as the corresponding query in $\text{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda)$.
 - (b) **Ciphertext Queries** $\mathcal{O}_{CT}$: The query is same as the corresponding query in $\text{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda)$.
 - (c) **Trapdoor Queries** $\mathcal{O}_{TD}$: The query is same as the corresponding query in $\text{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda)$.
3. **Challenge**: $\mathcal{A}$ chooses $\mathbf{tk}_0^*, \mathbf{tk}_1^*$ which have not been queried in **Phase 1** as two challenge trapdoor keywords, and sends them to $\mathcal{C}$. After that, $\mathcal{C}$ selects a random bit $\mu \in \{0, 1\}$ and invokes the **Trapdoor** $(pp, pk_S, sk_R, (\mathcal{S}^*, \xi^*), \mathbf{tk}_\mu^*)$ algorithm to obtain a challenge trapdoor TD_μ^* . Ultimately, $\mathcal{C}$ returns TD_μ^* to $\mathcal{A}$.
4. **Phase 2**: $\mathcal{A}$ continues to access $\mathcal{O}_{RK}$, $\mathcal{O}_{CT}$, and $\mathcal{O}_{TD}$ oracles, but subject to the restriction with neither $\mathbf{tk}_0^*$ nor $\mathbf{tk}_1^*$ can be queried.
5. **Guess**: $\mathcal{A}$ guesses a bit $\mu' \in \{0, 1\}$. If $\mu' = \mu$, we say that $\mathcal{A}$ wins this game.

We hereby define the advantage of $\mathcal{A}$ to win the above game $\text{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-IKGA}}(\lambda)$ as:

$$\text{Adv}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-IKGA}}(\lambda) = |\Pr[\mu' = \mu] - \frac{1}{2}|.$$

Definition 8. *Our RABAEKS primitive satisfies IND-IKGA security, if any PPT adversary wins the above game $\text{Exp}_{\text{RABAEKS}, \mathcal{A}}^{\text{IND-IKGA}}(\lambda)$ with a negligible advantage.*

5. Our Proposed RABAEKS Scheme

In this section, we propose an overview of our design. After that, we present the concrete construction of our RABAEKS scheme and analyze its correctness.

5.1. Design Rationale

In traditional PAEKS schemes, keyword ciphertexts are often stored in cloud servers, and data receivers submit a trapdoor to search for their interested keywords. Unfortunately, they can only support a single-receiver model, which loses scalability in cloud networks.

In 2023, Luo et al. [20] incorporated the idea of KP-ABE into PAEKS to construct a lattice-based ABAEKS scheme, thereby supporting the multi-receiver model. However, the existing approach presents three challenges to be considered. Firstly, in [20], the attribute is embedded in the keyword ciphertext (aka key-policy), which cannot provide attributes to each data receiver. Secondly, this scheme does not consider the receiver revocation, which significantly limits its effectiveness when the receiver's access permission changes dynamically. Thirdly, it is non-trivial to generate a valid keyword ciphertext and search trapdoor and match them correctly.

To solve the aforementioned challenges, we construct a ciphertext-policy solution by initializing an attribute set $\mathcal{ATT} := \{1, \dots, |att|\}$ and many attribute matrices $\mathbf{A}_a, \{\mathbf{A}_i\}_{i \in \mathcal{ATT}} \in \mathbb{Z}_q^{n \times m}$, and generate the receiver's secret key $\{\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))}\}_{j \in \mathcal{S}}$ utilized these attribute matrices and the **SampleBasisLeft** algorithm. Then, we set an access policy $(\mathcal{W}, t)$ into the keyword ciphertext, and use the Lagrange interpolation algorithm to achieve the receiver access control, thereby addressing the first challenge.

With regard to the second challenge, we implement the revocation list $\mathcal{R}$ utilizing the binary tree structure $\mathcal{T}$ and the **KUNodes** algorithm, subsequently revoking the access permission of a specific receiver.

For the third challenge, we embed the secret key $\mathbf{T}_{\mathbf{A}_S}$ of the data sender to authenticate the keyword $\mathbf{ck} = (ck_1, \dots, ck_{|kw|}) \in \{0, 1\}^{|kw|}$ through the **SampleLeft** algorithm. When calculating a trapdoor, it is necessary to find a correct basis for lattice $\Lambda_q^\perp(\mathbf{A}_0 | (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) | (\mathbf{A}_a + \mathbf{A}_j) | \mathbf{B}_{w_\tau})$ or $\Lambda_q^\perp((\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) | (\mathbf{A}_a + \mathbf{A}_j) | \mathbf{B}_{w_\tau})$, but we can only obtain a basis $\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))}$ for lattice $\Lambda_q^\perp(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))$ for $j \in \mathcal{S}$ during secret key generation. As a result, there is no suitable lattice basis available for the common lattice basis sampling algorithms (e.g., **SamplePre**, **SampleLeft**). Given a short basis for $\Lambda_q^\perp(\mathbf{A}')$, the **GenSamplePre** algorithm is able to sample a vector, where $\mathbf{A}'$ is the horizontal concatenation of any number of matrices in $(\mathbf{A}_0 | (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) | (\mathbf{A}_a + \mathbf{A}_j) | \mathbf{B}_{w_\tau})$, e.g. $(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))$. In this way, we innovatively invoke this algorithm to a valid trapdoor vector inputting the secret key $\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))}$, thereby addressing the third challenge.

Consequently, we can construct a lattice-based RABAEKS scheme that supports dynamic access control in a multi-receiver model, offering a more practical encrypted search scheme in cloud networks.

5.2. Our Concrete Construction

The design of our RABAEKS includes six phases: *System Initialization*, *Key Generation*, *Ciphertext Generation*, *Trapdoor Generation*, *Search Phase*, and *Ciphertext Update*.

5.2.1. System Initialization

The TA sets up this system by calling the **Setup**(λ) algorithm. A security parameter λ is input, and this algorithm outputs the public parameter pp and master key msk according to the following procedures. Firstly, the TA sets the system parameters $n, m, q, \sigma, |att|, l, |kw|, \mathcal{ATT}, \mathcal{U}, \mathcal{R}$, and calculates an attribute set $\mathcal{ATT} = \{1, \dots, |att|\}$. Then, to calculate the master secret key, the TA invokes $(\mathbf{A}_0, \mathbf{T}_{\mathbf{A}_0}) \leftarrow \text{TrapGen}(n, m, q)$ to obtain an uniformly matrix $\mathbf{A}_0 \in \mathbb{Z}_q^{n \times m}$ and a basis $\mathbf{T}_{\mathbf{A}_0} \in \mathbb{Z}^{m \times m}$ of $\Lambda_q^\perp(\mathbf{A}_0)$. After that, the TA selects three random matrices $\mathbf{A}_a \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\mathbf{A}_w \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\bar{\mathbf{A}} \xleftarrow{\$} \mathbb{Z}_q^{n \times n}$. For $i \in \mathcal{ATT}$, the TA chooses a random matrix $\mathbf{A}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$. For $i \in [|kw|]$, the TA selects a random matrix $\mathbf{W}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$. Due to realize the receiver revocation, the TA initializes a binary tree $\mathcal{T}$ and many matrices $\mathbf{B}_0 \in \mathbb{Z}_q^{n \times m}$, $\{\mathbf{U}_i\}_{i \in [1, |\mathcal{U}|-2]}$, $\{\mathbf{V}_i\}_{i \in [|\mathcal{U}|-1, 2|\mathcal{U}|-2]}$ defined in **Definition 5**. Finally, the TA returns the public parameter $pp := (n, m, q, \sigma, |att|, l, |kw|, \mathcal{ATT}, \mathcal{U}, \mathcal{R}, \mathbf{A}_0, \mathbf{A}_a, \mathbf{A}_w, \{\mathbf{A}_i\}_{i \in \mathcal{ATT}}, \{\mathbf{W}_i\}_{i \in [|kw|]}, \mathbf{B}_0, \{\mathbf{U}_i\}_{i \in [1, |\mathcal{U}|-2]}, \{\mathbf{V}_i\}_{i \in [|\mathcal{U}|-1, 2|\mathcal{U}|-2]}, \mathcal{T})$, and the master key $msk := \mathbf{T}_{\mathbf{A}_0}$.

5.2.2. Key Generation

The TA calculates the public and secret key of the data sender and the secret key of the data receiver by calling the **KeyGen_S**(pp) and **KeyGen_R**($pp, \mathcal{S}, msk$) algorithms, respectively.

For the **KeyGen_S**(pp) algorithm, after inputting a public parameter pp , this algorithm outputs the public key pk_S and secret key sk_S with the data sender through the **TrapGen** algorithm. TA first invokes $(\mathbf{A}_S, \mathbf{T}_{\mathbf{A}_S}) \leftarrow \text{TrapGen}(n, m, q)$ to obtain an uniformly matrix $\mathbf{A}_S \in \mathbb{Z}_q^{n \times m}$ and a basis $\mathbf{T}_{\mathbf{A}_S} \in \mathbb{Z}^{m \times m}$ for $\Lambda_q^\perp(\mathbf{A}_S)$, and then sends the public key $pk_S := \mathbf{A}_S$ and the secret key $sk_S := \mathbf{T}_{\mathbf{A}_S}$ to the data sender.

For the $\mathbf{KeyGen}_R(pp, \mathcal{S}, msk)$ algorithm, a public parameter pp , a receiver attribute set $\mathcal{S} \subseteq \mathcal{ATT}$, and a master key msk are input, and the TA calls the **SampleBasisLeft** algorithm to output the secret key sk_R with the data receiver. For $j \in \mathcal{S}$, the TA invokes $\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))} \leftarrow \mathbf{SampleBasisLeft}(\mathbf{A}_0, \mathbf{A}_a + \mathbf{A}_j, \mathbf{T}_{\mathbf{A}_0}, \sigma)$ to obtain a basis $\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))} \in \mathbb{Z}^{2m \times 2m}$ for $\Lambda_q^\perp(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))$. Then, the TA sends the secret key $sk_R := \{\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))}\}_{j \in \mathcal{S}}$ to the data receiver.

5.2.3. Ciphertext Generation

The data sender calculates a keyword ciphertext by calling the **Encrypt** $(pp, pk_S, sk_S, (\mathcal{W}, t), \mathbf{ck})$ algorithm. A public parameter pp , the public and secret keys (pk_S, sk_S) with the data sender, an access policy $(\mathcal{W}, t)$ where $\mathcal{W} \subset \mathcal{ATT}$, and a keyword $\mathbf{ck} \in \{0, 1\}^{|\mathcal{kw}|}$ are input, and this algorithm outputs a keyword ciphertext CT according to the **Algorithm 2**.

5.2.4. Trapdoor Generation

The data receiver $\xi \in \mathcal{U}$ calculates a search trapdoor by calling the **Trapdoor** $(pp, pk_S, sk_R, (\mathcal{S}, \xi), \mathbf{tk})$ algorithm. A public parameter pp , the public key pk_S with the data sender, the secret keys sk_R and the attribute set $\mathcal{S}$ with the data receiver ξ , and a keyword $\mathbf{tk} \in \{0, 1\}^{|\mathcal{kw}|}$ are input, and this algorithm outputs a search trapdoor TD according to the **Algorithm 3**.

5.2.5. Search Phase

According to the **Algorithm 4**, the cloud server executes the **Search** $(pp, (\mathcal{S}, \xi), (\mathcal{W}, t), \text{CT}, \text{TD})$ algorithm. A public parameter pp , the attribute set $\mathcal{S}$ with data receiver ξ , the access policy $(\mathcal{W}, t)$, the keyword ciphertext CT, and the trapdoor TD are input, if CT and TD corresponds the same keyword, this algorithm outputs 1. Otherwise, this algorithm outputs 0.

5.2.6. Ciphertext Update

The cloud server updates the ciphertext with the change of revocation list by calling the **CTUpdate** $(pp, \mathcal{R}', \text{CT})$ algorithm. A public parameter pp , the updated revocation list $\mathcal{R}'$, and the keyword ciphertext CT are input, and this algorithm outputs an updated keyword ciphertext CT' according to the **Algorithm 5**.

Algorithm 2 Encrypt

Input: A public parameter pp , a public key of sender pk_S , a secret key of sender sk_S , an access policy $(\mathcal{W}, t)$, and a keyword $\mathbf{ck}$.

Output: The keyword ciphertext CT.

- 1 Parse the keyword $\mathbf{ck} = (ck_1, \dots, ck_{|kw|}) \in \{0, 1\}^{|kw|}$.
 - 2 Select a random vector $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$.
 - 3 Select many noise vectors $\mathbf{x}_0 \xleftarrow{\$} \Psi_\alpha^m$, $\mathbf{x}_1 \xleftarrow{\$} \Psi_\alpha^m$, $\bar{\mathbf{x}} \xleftarrow{\$} \Psi_\alpha^n$ and $\mathbf{x}_M \xleftarrow{\$} \Psi_\alpha^m$.
 - 4 Calculate three vectors $\mathbf{c}_0 = \mathbf{A}_0^\top \mathbf{s} + \mathbf{x}_0 \in \mathbb{Z}_q^m$, $\mathbf{c}_1 = (\mathbf{A}_w + \sum_{i=1}^{|kw|} ck_i \mathbf{W}_i)^\top \mathbf{s} + \mathbf{x}_1 \in \mathbb{Z}_q^m$, and $\bar{\mathbf{c}} = \bar{\mathbf{A}}^\top \mathbf{s} + \bar{\mathbf{x}} \in \mathbb{Z}_q^n$.
 - 5 Invoke $\mathbf{c}_2 \leftarrow \text{SampleLeft}(\mathbf{A}_S, \mathbf{A}_w + \sum_{i=1}^{|kw|} ck_i \mathbf{W}_i, \mathbf{T}_{\mathbf{A}_S}, \bar{\mathbf{c}}, \sigma)$ to obtain a vector $\mathbf{c}_2 \in \mathbb{Z}_q^{2m}$, s.t. $(\mathbf{A}_S \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} ck_i \mathbf{W}_i)) \mathbf{c}_2 = \bar{\mathbf{c}} \bmod q$.
 - 6 **for** $i \in \mathcal{W}$ **do**
 - 7 Select a noise vector $\mathbf{x}_i \xleftarrow{\$} \Psi_\alpha^m$.
 - 8 Calculate a vector $\mathbf{c}_i = (\mathbf{A}_a + \mathbf{A}_i)^\top \mathbf{s} + \mathbf{x}_i \in \mathbb{Z}_q^m$.
 - 9 **end**
 - 10 **for** $k \in \text{KUNodes}(\mathcal{T}, \mathcal{R})$ **do**
 - 11 Select a noise vector $\hat{\mathbf{x}}_k \xleftarrow{\$} \Psi_\alpha^m$.
 - 12 Calculate a vector $\hat{\mathbf{c}}_k = \mathbf{B}_k^\top \mathbf{s} + \hat{\mathbf{x}}_k \in \mathbb{Z}_q^m$.
 - 13 **end**
 - 14 **return** the ciphertext $\text{CT} := (\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \{\mathbf{c}_i\}_{i \in \mathcal{W}}, \{\hat{\mathbf{c}}_k\}_{k \in \text{KUNodes}(\mathcal{T}, \mathcal{R})})$ corresponding to the keyword $\mathbf{ck}$.
-

5.3. Correctness Analysis and Parameter Setting

To prove the correctness of our RABAEKS scheme, we suppose the public and secret key of data sender $(pk_S, sk_S) = (\mathbf{A}_S, \mathbf{T}_{\mathbf{A}_S})$, the secret key of data receiver $sk_R = \{\mathbf{T}_{(\mathbf{A}_0 \mid (\mathbf{A}_a + \mathbf{A}_j))}\}_{j \in \mathcal{S}}$, a ciphertext keyword $\mathbf{ck}$ and its ciphertext $\text{CT} := (\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \{\mathbf{c}_i\}_{i \in \mathcal{W}}, \{\hat{\mathbf{c}}_k\}_{k \in \text{KUNodes}(\mathcal{T}, \mathcal{R})})$, the trapdoor keyword $\mathbf{tk}$ and corresponding search trapdoor $\text{TD} := (\mathbf{t}_0, \{\mathbf{t}_i\}_{i \in \mathcal{S}})$. When $|\mathcal{S} \cap \mathcal{W}| \geq t$ and $\mathbf{ck} = \mathbf{tk}$, we set $\mathcal{J} \subset \mathcal{S} \cap \mathcal{W}$ and $\gamma = \text{KUNodes}(\mathcal{T}, \mathcal{R}) \cap \omega(\xi)$, where $|\mathcal{J}| = t$.

If γ is a non-leaf node, $\mathbf{c}'_\gamma = \mathbf{V}_\gamma^\top (\mathbf{U}_\gamma^\top)^{-1} \hat{\mathbf{c}}_\gamma = \mathbf{V}_{\omega_\tau}^\top (\mathbf{U}_\gamma^\top)^{-1} (\mathbf{B}_\gamma^\top \mathbf{s} + \hat{\mathbf{x}}_\gamma) = \mathbf{V}_{\omega_\tau}^\top (\mathbf{U}_\gamma^\top)^{-1} (\mathbf{U}_\gamma^\top \mathbf{B}_0^\top \mathbf{s} + \hat{\mathbf{x}}_\gamma) = \mathbf{V}_{\omega_\tau}^\top \mathbf{B}_0^\top \mathbf{s} + \mathbf{V}_{\omega_\tau}^\top (\mathbf{U}_\gamma^\top)^{-1} \hat{\mathbf{x}}_\gamma = \mathbf{B}_{\omega_\tau}^\top \mathbf{s} + \mathbf{x}'_{\omega_\tau}$, where $\mathbf{x}'_{\omega_\tau} = \mathbf{V}_{\omega_\tau}^\top (\mathbf{U}_\gamma^\top)^{-1} \hat{\mathbf{x}}_\gamma$. If γ is a leaf node, $\mathbf{c}'_\gamma = \hat{\mathbf{c}}_{\omega_\tau} = \mathbf{B}_{\omega_\tau}^\top \mathbf{s} + \mathbf{x}'_{\omega_\tau}$, where $\mathbf{x}'_{\omega_\tau} = \hat{\mathbf{x}}_{\omega_\tau}$.

Algorithm 3 Trapdoor

Input: A public parameter pp , a public key of sender pk_S , a secret key of receiver sk_R , an attribute set $\mathcal{S}$, the receiver number ξ , and a keyword $\mathbf{tk}$.

Output: The search trapdoor TD.

- 1 Parse the keyword $\mathbf{tk} = (tk_1, \dots, tk_{|kw|}) \in \{0, 1\}^{|kw|}$.
 - 2 Select a random vector $\mathbf{s}' \xleftarrow{\$} \mathbb{Z}_q^n$.
 - 3 Select some random matrices $\{\mathbf{R}'_i\}_{i \in [|kw|]} \xleftarrow{\$} \{-1, 1\}^{m \times m}$, and two noise vectors $\mathbf{x}'_0 \xleftarrow{\$} \Psi_\alpha^m$ and $\bar{\mathbf{x}}' \xleftarrow{\$} \Psi_\alpha^n$.
 - 4 Calculate two vectors $\mathbf{t}_0 = (\mathbf{A}_S \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i))^\top \mathbf{s}' + \begin{pmatrix} \mathbf{x}'_0 \\ \mathbf{R}'^\top \mathbf{x}'_0 \end{pmatrix} \in \mathbb{Z}_q^{2m}$,
and $\bar{\mathbf{t}} = \bar{\mathbf{A}}\mathbf{s}' + \bar{\mathbf{x}}' := (\bar{t}_1, \dots, \bar{t}_n) \in \mathbb{Z}_q^n$, where $\mathbf{R}' = \sum_{i=1}^{|kw|} ck_i \mathbf{R}'_i \in \mathbb{Z}^{m \times m}$.
 - 5 Calculate a set $\omega(\xi) = \{w_0, \dots, w_\tau\}$.
 - 6 **for** $j \in [n]$ **do**
 - 7 Select many polynomials $p_j(x) \in \mathbb{Z}_q[x]$, such that $p_j(0) = \bar{t}_j \in \mathbb{Z}_q$.
 - 8 **end**
 - 9 **for** $j \in \mathcal{S}$ **do**
 - 10 Set the vector $\hat{\mathbf{t}}_j = (p_1(j), \dots, p_n(j))^\top \in \mathbb{Z}_q^n$.
 - 11 Invoke $\mathbf{t}_j \leftarrow \text{GenSamplePre}(\mathbf{A}_0 \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) \mid (\mathbf{A}_a + \mathbf{A}_j) \mid \mathbf{B}_{w_\tau}, \mathbf{T}_{(\mathbf{A}_0 \mid (\mathbf{A}_a + \mathbf{A}_j))}, \{1, 3\}, \hat{\mathbf{t}}_j, \sigma)$ to obtain a vector $\mathbf{t}_j \in \mathbb{Z}_q^{4m}$, s.t. $(\mathbf{A}_0 \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) \mid (\mathbf{A}_a + \mathbf{A}_j) \mid \mathbf{B}_{w_\tau})\mathbf{t}_j = \hat{\mathbf{t}}_j \bmod q$.
 - 12 **end**
 - 13 **return** the trapdoor TD := $(\mathbf{t}_0, \{\mathbf{t}_j\}_{j \in \mathcal{S}})$ corresponding to the keyword $\mathbf{tk}$.
-

Based on the aforementioned conditions, we have:

$$\begin{aligned}
r &= \mathbf{c}_2^\top \mathbf{t}_0 - \sum_{j \in \mathcal{J}} L_j(\mathbf{c}_0^\top \mid \mathbf{c}_1^\top \mid \mathbf{c}_j^\top \mid \mathbf{c}'_\gamma^\top) \mathbf{t}_j \\
&= \mathbf{c}_2^\top (\mathbf{A}_S \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i))^\top \mathbf{s}' + \mathbf{c}_2^\top \begin{pmatrix} \mathbf{x}'_0 \\ \mathbf{R}'^\top \mathbf{x}'_0 \end{pmatrix} \\
&\quad - \sum_{j \in \mathcal{J}} L_j((\mathbf{s}^\top \mathbf{A}_0 + \mathbf{x}_0^\top) \mid (\mathbf{s}^\top (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) + \mathbf{x}_1^\top) \\
&\quad \mid (\mathbf{s}^\top (\mathbf{A}_a + \mathbf{A}_j) + \mathbf{x}_j^\top) \mid (\mathbf{s}^\top \mathbf{B}_{w_\tau} + \mathbf{x}'_{w_\tau}^\top)) \mathbf{t}_j
\end{aligned}$$

Algorithm 4 Search

Input: A public parameter pp , an attribute set $\mathcal{S}$, the receiver number ξ , an access policy $(\mathcal{W}, t)$, the keyword ciphertext CT, and the trapdoor TD.

Output: The search result 1 or 0.

```

1 if  $|\mathcal{S} \cap \mathcal{W}| < t$  or  $\text{KUNodes}(\mathcal{T}, \mathcal{R}) \cap \omega(\xi) = \emptyset$  then
2   | return 0.
3 end
4 else
5   | Select a set  $\mathcal{J} \subset \mathcal{S} \cap \mathcal{W}$ , where  $|\mathcal{J}| = t$ , and set  $\gamma = \text{KUNodes}(\mathcal{T}, \mathcal{R}) \cap \omega(\xi)$ 
6   | if  $\gamma$  is a non-leaf node then
7     | Calculate  $\mathbf{c}'_\gamma = \mathbf{V}_{\omega_\tau}^\top (\mathbf{U}_\gamma^\top)^{-1} \hat{\mathbf{c}}_\gamma$ .
8   | end
9   | if  $\gamma$  is a leaf node then
10    | Calculate  $\mathbf{c}'_\gamma = \hat{\mathbf{c}}_{\omega_\tau}$ .
11   | end
12 end
13 Calculate a number  $r = \mathbf{c}_2^\top \mathbf{t}_0 - \sum_{j \in \mathcal{J}} L_j(\mathbf{c}_0^\top \mid \mathbf{c}_1^\top \mid \mathbf{c}_j^\top \mid \mathbf{c}'_\gamma{}^\top) \mathbf{t}_j \in \mathbb{Z}_q$ , where
    
$$L_j = \frac{\prod_{i \in \mathcal{J}, i \neq j} -i}{\prod_{i \in \mathcal{J}, i \neq j} (j-i)}.$$

14 if  $|r - \lfloor \frac{q}{2} \rfloor| \leq \lfloor \frac{q}{4} \rfloor$  then
15   | return 1.
16 end
17 else
18   | return 0.
19 end

```

$$\begin{aligned}
&= \bar{\mathbf{c}}^\top \mathbf{s}' - \sum_{j \in \mathcal{J}} L_j \mathbf{s}^\top (\mathbf{A}_0 \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) \mid (\mathbf{A}_a + \mathbf{A}_j) \mid \mathbf{B}_{\omega_\tau}) \mathbf{t}_j \\
&\quad + \mathbf{c}_2^\top \begin{pmatrix} \mathbf{x}'_0 \\ \mathbf{R}'^\top \mathbf{x}'_0 \end{pmatrix} - \sum_{j \in \mathcal{J}} L_j (\mathbf{x}_0^\top \mid \mathbf{x}_1^\top \mid \mathbf{x}_j^\top \mid \mathbf{x}'_{\omega_\tau}{}^\top) \mathbf{t}_j \\
&= \bar{\mathbf{c}}^\top \mathbf{s}' - \sum_{j \in \mathcal{J}} L_j \mathbf{s}^\top \hat{\mathbf{t}}_j + \mathbf{x}_L \\
&= \bar{\mathbf{c}}^\top \mathbf{s}' - \mathbf{s}^\top \bar{\mathbf{t}} + \mathbf{x}_L = \mathbf{s}^\top \bar{\mathbf{A}} \mathbf{s}' + \bar{\mathbf{x}}^\top \mathbf{s}' - \mathbf{s}^\top \bar{\mathbf{A}} \mathbf{s}' - \mathbf{s}^\top \bar{\mathbf{x}}' + \mathbf{x}_L \\
&= \mathbf{x}_L + \bar{\mathbf{x}}^\top \mathbf{s}' - \mathbf{s}^\top \bar{\mathbf{x}}',
\end{aligned}$$

Algorithm 5 CTUpdate

Input: A public parameter pp , the updated revocation list $\mathcal{R}'$, and the keyword ciphertext CT.

Output: An updated keyword ciphertext CT'.

```
1 for  $k' \in \text{KUNodes}(\mathcal{T}, \mathcal{R}')$  do
2   Set  $\zeta$  is an arbitrary non-leaf node in  $\mathcal{R}$ .
3   if  $k'$  is a non-leaf node then
4     Calculate a vector  $\mathbf{c}'_{k'} = \mathbf{U}_{k'}^\top (\mathbf{U}_\zeta^\top)^{-1} \mathbf{c}_\zeta \in \mathbb{Z}_q^n$ .
5   end
6   else
7     Calculate a vector  $\mathbf{c}'_{k'} = \mathbf{V}_{k'}^\top (\mathbf{U}_\zeta^\top)^{-1} \mathbf{c}_\zeta \in \mathbb{Z}_q^n$ .
8   end
9 end
10 return an updated keyword ciphertext  $\text{CT}' := (\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \{\mathbf{c}_i\}_{i \in \mathcal{W}}, \{\mathbf{c}'_{k'}\}_{k' \in \text{KUNodes}(\mathcal{T}, \mathcal{R}')}).$ 
```

where $\mathbf{x}_L = \mathbf{c}_2^\top \begin{pmatrix} \mathbf{x}'_0 \\ \mathbf{R}'^\top \mathbf{x}'_0 \end{pmatrix} - \sum_{j \in \mathcal{J}} L_j(\mathbf{x}_0^\top \mid \mathbf{x}_1^\top \mid \mathbf{x}_j^\top \mid \mathbf{x}'_{\omega_\tau}) \mathbf{t}_j$.

If $|r| = |\mathbf{x}_L + \bar{\mathbf{x}}^\top \mathbf{s}' - \mathbf{s}^\top \bar{\mathbf{x}}'| < \frac{q}{5}$ [52], the cloud server will return 1, which represents that the ciphertext and trapdoor correspond to the same keyword.

Based on this, the involved parameters are set as follows:

- $|r| = q\sigma m\alpha\omega(\sqrt{\log m}) + 2q\sigma m\alpha\omega(\sqrt{\log 4m}) + 2q\sigma n\alpha\omega(\sqrt{\log n}) + O(\sigma m^{\frac{3}{2}}) + O(\sigma n) < \frac{q}{5}$ for a correct search in RABAEKS.
- $m \geq \lceil 2n \log q \rceil$ for TrapGen algorithm.
- $\sigma \geq 4m \cdot \omega(\log 4m)$ for SampleBasisLeft, SampleLeft and GenSamplePre algorithms.
- $\alpha q > 2\sqrt{n}$ for LWE hardness.

6. Security Analysis

In this section, we rigorously prove the security of our scheme, which covers IND-CKA and IND-IKGA security.

Theorem 1. *Assume that the $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness holds, our proposed lattice-based RABAEKS scheme satisfies IND-CKA security in the ROM.*

For any PPT adversary $\mathcal{A}$, if $\mathcal{A}$ can compromise our scheme with a non-negligible advantage ϵ_1 , then a PPT challenger $\mathcal{C}$ can be constructed to solve the $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness with a non-negligible probability.

Proof 1. Assume that there exists a PPT adversary $\mathcal{A}$ that has the ability to break the IND-CKA security with a non-negligible advantage ϵ_1 , we can construct a challenger $\mathcal{C}$ that solves the decisional $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness with the same advantage. In the initialization phase, $\mathcal{A}$ sends a challenge access structure $(\mathcal{W}^*, t^*)$ and a revocation list $\mathcal{R}^*$ to $\mathcal{C}$, where $t^* \in [1, |\text{att}|]$. Let $\mathcal{ATT} = \{1, \dots, |\text{att}|\}$, such that $\mathcal{W}^* \subset \mathcal{ATT}$, the interaction between $\mathcal{A}$ and $\mathcal{C}$ is showed as follows.

Setup: The challenger $\mathcal{C}$ samples $(\mathbf{A}_0, \mathbf{b}_0)$, $(\mathbf{A}_w, \mathbf{b}_w)$, $\{(\mathbf{W}_i, \mathbf{u}_i)\}_{i \in [|\mathcal{kw}|]}$, and $\{(\mathbf{B}_i, \mathbf{v}_i)\}_{i \in [1, 2|\mathcal{U}|-2]}$ from the oracle $\mathcal{O}$, where $\mathcal{O} = \mathcal{O}_{\text{LWE}}$ or $\mathcal{O} = \mathcal{O}'_{\text{LWE}}$ meaning that these tuples are pseudo-random or random, respectively. Then, $\mathcal{C}$ invokes $(\mathbf{A}_a, \mathbf{T}_{\mathbf{A}_a}) \leftarrow \text{TrapGen}(n, m, q)$ to generate a matrix $\mathbf{A}_a \in \mathbb{Z}_q^{n \times m}$ and a basis $\mathbf{T}_{\mathbf{A}_a} \in \mathbb{Z}^{m \times m}$ for $\Lambda_q^\perp(\mathbf{A}_a)$. After that, $\mathcal{C}$ selects several random matrices $\overline{\mathbf{A}} \xleftarrow{\$} \mathbb{Z}_q^{n \times n}$, $\mathbf{R}_i^* \in \{-1, 1\}^{m \times m}$. For $i \in \mathcal{ATT}$, if $i \in \mathcal{W}^*$, $\mathcal{C}$ computes $\mathbf{A}_i = \mathbf{A}_0 \mathbf{R}_i^* - \mathbf{A}_a \in \mathbb{Z}_q^{n \times m}$. Otherwise, $\mathcal{C}$ computes $\mathbf{A}_i = \mathbf{A}_0 \mathbf{R}_i^* \in \mathbb{Z}_q^{n \times m}$. Finally, $\mathcal{C}$ sets $\{\mathbf{U}_i\}_{i \in [1, |\mathcal{U}|-2]} = \{\mathbf{B}_i\}_{i \in [1, |\mathcal{U}|-2]}$, $\{\mathbf{V}_i\}_{i \in [|\mathcal{U}|-1, 2|\mathcal{U}|-2]} = \{\mathbf{B}_i\}_{i \in [|\mathcal{U}|-1, 2|\mathcal{U}|-2]}$ and sends the public parameter $pp = (n, m, q, \sigma, |\text{att}|, l, |\mathcal{kw}|, \mathcal{ATT}, \mathcal{U}, \mathcal{R}, \mathbf{A}_0, \mathbf{A}_a, \mathbf{A}_w, \overline{\mathbf{A}}, \{\mathbf{A}_i\}_{i \in \mathcal{ATT}}, \{\mathbf{W}_i\}_{i \in [|\mathcal{kw}|]}, \mathbf{B}_0, \{\mathbf{U}_i\}_{i \in [1, |\mathcal{U}|-2]}, \{\mathbf{V}_i\}_{i \in [|\mathcal{U}|-1, 2|\mathcal{U}|-2]}, \mathcal{T})$ to $\mathcal{A}$.

Phase 1: The adversary $\mathcal{A}$ executes these following queries adaptively.

- **Secret Key Queries $\mathcal{O}_{RK}$:** If $\mathcal{S}$ meets the access policy $(\mathcal{W}^*, t^*)$, $\mathcal{C}$ returns $\perp$ to $\mathcal{A}$. Otherwise, $\mathcal{C}$ invokes $\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))} \leftarrow \text{SampleBasisRight}(\mathbf{A}_0, \mathbf{A}_a, \mathbf{R}_j^*, \mathbf{T}_{\mathbf{A}_a}, \sigma)$ to obtain a basis $\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))} \in \mathbb{Z}^{2m \times 2m}$ for $\Lambda_q^\perp(\mathbf{A}_0 | (\mathbf{A}_0 \mathbf{R}_j^* + \mathbf{A}_a)) = \Lambda_q^\perp(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))$, for $j \in \mathcal{S}$. Finally, $\mathcal{C}$ returns the secret key $sk_R := \{\mathbf{T}_{(\mathbf{A}_0 | (\mathbf{A}_a + \mathbf{A}_j))}\}_{j \in \mathcal{S}}$ to $\mathcal{A}$.
- **Ciphertext Queries $\mathcal{O}_{CT}$:** After $\mathcal{A}$ inputs $((\mathcal{W}, t), \mathbf{ck})$, $\mathcal{C}$ executes $\text{Encrypt}(pp, pk_S, sk_S, (\mathcal{W}, t), \mathbf{ck})$ to generate the ciphertext CT with the keyword $\mathbf{ck}$, and then returns it to $\mathcal{A}$.
- **Trapdoor Queries $\mathcal{O}_{TD}$:** After $\mathcal{A}$ inputs $((\mathcal{S}, \xi), \mathbf{tk})$, $\mathcal{C}$ executes the following procedures according to different conditions.

1. If $\mathcal{S}$ does not meet $(\mathcal{W}^*, t^*)$ and $\xi \in \mathcal{R}^*$, where $|\mathcal{S} \cap \mathcal{W}^*| < t^*$. $\mathcal{C}$ calculates a secret key sk_R with data receiver by executing $\mathcal{O}_{RK}$, and

calculates $\mathbf{t}_0$ as same as the process in **Trapdoor**($pp, pk_S, sk_R, (\mathcal{S}, \xi), \mathbf{tk}$). Then, $\mathcal{C}$ selects a subset $\mathcal{J} \subseteq \mathcal{S}$ such that $|\mathcal{J}| = t$. For $j \in \mathcal{J}$, $\mathcal{C}$ samples a vector $\mathbf{t}_j \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}_q^{4m}, \sigma}$, and calculates $\hat{\mathbf{t}}_j = (\mathbf{A}_0 \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) \mid (\mathbf{A}_a + \mathbf{A}_j) \mid \mathbf{B}_{w_\tau}) \mathbf{t}_j \bmod q$. Consequently, the pairing $\{\hat{\mathbf{t}}, \{\hat{\mathbf{t}}_j\}\}$ can be obtained. Through the Lagrange interpolation, the polynomials $p_1(x), \dots, p_n(x)$ can be recovered. For $j \in \mathcal{S}$, $\hat{\mathbf{t}}_j = (p_1(j), \dots, p_n(j))^\top$, $\mathcal{C}$ invokes $\mathbf{t}_j \leftarrow \text{GenSamplePre}(\mathbf{A}_0 \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) \mid (\mathbf{A}_a + \mathbf{A}_j) \mid \mathbf{B}_{w_\tau}, \mathbf{T}_{(\mathbf{A}_0 \mid (\mathbf{A}_a + \mathbf{A}_j))}, \{1, 3\}, \hat{\mathbf{t}}_j, \sigma)$ to obtain a vector $\mathbf{t}_j \in \mathbb{Z}_q^{4m}$. Finally, $\mathcal{C}$ returns the trapdoor $\text{TD} = (\mathbf{t}_0, \{\mathbf{t}_j\}_{j \in \mathcal{S}})$ to $\mathcal{A}$.

2. If $\mathcal{S}$ meets $(\mathcal{W}^*, t^*)$ and $\xi \in \mathcal{R}^*$, $\mathcal{C}$ calculates a vector $\mathbf{t}_0$ firstly. Then, $\mathcal{C}$ samples $\{\mathbf{t}_j\}_{j \in \mathcal{S}} \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}_q^{4m}, \sigma}$ randomly, and returns $\text{TD} = (\mathbf{t}_0, \{\mathbf{t}_j\}_{j \in \mathcal{S}})$ to $\mathcal{A}$.
3. If $\mathcal{S}$ does not meet $(\mathcal{W}^*, t^*)$ and $\xi \notin \mathcal{R}^*$, $\mathcal{C}$ calculates a vector $\mathbf{t}_0$ firstly, and then invokes $\mathbf{t}_j \leftarrow \text{GenSamplePre}(\mathbf{A}_0 \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_i \mathbf{W}_i) \mid (\mathbf{A}_a + \mathbf{A}_j) \mid \mathbf{B}_{w_\tau}, \mathbf{T}_{(\mathbf{A}_0 \mid (\mathbf{A}_a + \mathbf{A}_j))}, \{1, 3\}, \hat{\mathbf{t}}_j, \sigma)$ to obtain a vector $\mathbf{t}_j \in \mathbb{Z}_q^{4m}$, where $\hat{\mathbf{t}}_j = (p_1(j), \dots, p_n(j))^\top \in \mathbb{Z}_q^n$ and $p_1(x), \dots, p_n(x) \in \mathbb{Z}_q[x]$ such that $(p_1(0), \dots, p_n(0))^\top = \hat{\mathbf{t}}$ for $j \in \mathcal{S}$. Then, $\mathcal{C}$ returns $\text{TD} = (\mathbf{t}_0, \{\mathbf{t}_j\}_{j \in \mathcal{S}})$ to $\mathcal{A}$.
4. If $\mathcal{S}$ meets $(\mathcal{W}^*, t^*)$ and $\xi \notin \mathcal{R}^*$, $\mathcal{C}$ returns $\perp$ to $\mathcal{A}$.

Challenge: $\mathcal{A}$ selects two keywords $\mathbf{ck}_0^*, \mathbf{ck}_1^*$ which have not been queried in **Phase 1**, and sends them to $\mathcal{C}$. Then, $\mathcal{C}$ selects a bit $\mu \in \{0, 1\}$ randomly, and encrypts $\mathbf{ck}_\mu^*$ with the access policy $(\mathcal{W}^*, t^*)$. Specially, $\mathcal{C}$ calculates $\mathbf{c}_0^* = \mathbf{b}_0$, $\mathbf{c}_1^* = \mathbf{b}_w + \sum_{i=1}^{|kw|} ck_i^* \mathbf{u}_i$, and samples a random vector $\mathbf{c}_2^* \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}_q^{2m}, \sigma}$. For $i \in \mathcal{W}^*$, $\mathcal{C}$ calculates $\mathbf{c}_i^* = (\mathbf{R}_i^*)^\top \mathbf{b}_0$. For $k \in \text{KUNodes}(\mathcal{T}, \mathcal{R}^*)$, $\hat{\mathbf{c}}_k^* = \mathbf{v}_j$, where $j \in [1, 2|\mathcal{U}| - 2]$. Finally, $\mathcal{C}$ returns $\text{CT}_\mu^* = (\mathbf{c}_0^*, \mathbf{c}_1^*, \mathbf{c}_2^*, \{\mathbf{c}_i^*\}_{i \in \mathcal{W}^*}, \{\hat{\mathbf{c}}_k^*\}_{k \in \text{KUNodes}(\mathcal{T}, \mathcal{R}^*)})$ to $\mathcal{A}$.

Phase 2: The adversary $\mathcal{A}$ continues to query from $\mathcal{O}_{RK}$, $\mathcal{O}_{CT}$, and $\mathcal{O}_{TD}$ as same as in **Phase 1** except that the $((\mathcal{S}, \xi), \mathbf{tk})$ could be queried in $\mathcal{O}_{TD}$ when $\mathcal{S}$ meets $(\mathcal{W}^*, t^*)$ and $\mathbf{tk} \notin (\mathbf{ck}_0^*, \mathbf{ck}_1^*)$.

Guess: The adversary $\mathcal{A}$ outputs $\mu' \in \{0, 1\}$. If $\mu' = \mu$, $\mathcal{A}$ wins this game and $\mathcal{C}$ outputs $\mathcal{O} = \mathcal{O}_{LWE}$. Otherwise, $\mathcal{C}$ outputs $\mathcal{O} = \mathcal{O}'_{LWE}$.

Analysis: If $\mu' = \mu$, $\mathcal{C}$ outputs $\mathcal{O} = \mathcal{O}_{LWE}$ meaning that $(\mathbf{A}_0, \mathbf{b}_0), (\mathbf{A}_w, \mathbf{b}_w), \{(\mathbf{W}_i, \mathbf{u}_i)\}_{i \in [|kw|]}$ and $\{(\mathbf{B}_i, \mathbf{v}_i)\}_{i \in [1, 2|\mathcal{U}| - 2]}$ are sampled from $\mathcal{O}_{LWE}$ such that $\mathbf{b}_0 = \mathbf{A}_0^\top \mathbf{s} + \mathbf{x}_0$, $\mathbf{b}_w = \mathbf{A}_w^\top \mathbf{s} + \mathbf{x}_w$, $\mathbf{u}_i = \mathbf{W}_i^\top \mathbf{s} + \mathbf{x}_i'$ and $\mathbf{v}_i = \mathbf{B}_i^\top \mathbf{s} + \mathbf{x}_i''$,

where $\mathbf{s} \in \mathbb{Z}_q^n$ is a secret vector and $\mathbf{x}_0 \leftarrow \Psi_\alpha^m$, $\mathbf{x}_w \leftarrow \Psi_\alpha^m$, $\{\mathbf{x}'_i\}_{i \in [kw]} \leftarrow \Psi_\alpha^m$ and $\{\mathbf{x}''_i\}_{i \in [1, 2|\mathcal{U}|-2]} \leftarrow \Psi_\alpha^m$ are noise vectors. Then, we can obtain the following equations:

$$\begin{aligned}
\mathbf{c}_0^* &= \mathbf{b}_0 = \mathbf{A}_0^\top \mathbf{s} + \mathbf{x}_0 \\
\mathbf{c}_1^* &= \mathbf{b}_w + \sum_{i=1}^{|kw|} ck_i^* \mathbf{u}_i = (\mathbf{A}_w + \sum_{i=1}^{|kw|} ck_i^* \mathbf{W}_i)^\top \mathbf{s} + (\mathbf{x}_w + \sum_{i=1}^{|kw|} ck_i^* \mathbf{x}_i) \\
&:= (\mathbf{A}_w + \sum_{i=1}^{|kw|} ck_i^* \mathbf{W}_i)^\top \mathbf{s} + \mathbf{x}_1 \\
\mathbf{c}_2^* &\stackrel{\$}{\leftarrow} \mathcal{D}_{\mathbb{Z}^{2m}, \sigma} \\
\mathbf{c}_i^* &= (\mathbf{R}_i^*)^\top \mathbf{b}_0 := (\mathbf{A}_a + \mathbf{A}_i)^\top \mathbf{s} + \mathbf{x}_i, i \in \mathcal{W}^* \\
\hat{\mathbf{c}}_k^* &= \mathbf{v}_k = \mathbf{B}_k^\top \mathbf{s} + \hat{\mathbf{x}}_k, k \in \text{KUNodes}(\mathcal{T}, \mathcal{R}^*)
\end{aligned}$$

Consequently, $\text{CT}_\mu^* = (\mathbf{c}_0^*, \mathbf{c}_1^*, \mathbf{c}_2^*, \{\mathbf{c}_i^*\}_{i \in \mathcal{W}^*}, \{\hat{\mathbf{c}}_k^*\}_{k \in \text{KUNodes}(\mathcal{T}, \mathcal{R}^*)})$ is a valid ciphertext, and assume that $\mathcal{A}$ can compromise our RABAEKS scheme with advantage ϵ_1 , thus $\Pr[\mu' = \mu \mid \mathcal{O} = \mathcal{O}_{LWE}] = \frac{1}{2} + \epsilon_1$. On the other hand, $\mathcal{C}$ outputs $\mathcal{O} = \mathcal{O}'_{LWE}$ meaning that $(\mathbf{A}_0, \mathbf{b}_0), (\mathbf{A}_w, \mathbf{b}_w), \{(\mathbf{W}_i, \mathbf{u}_i)\}_{i \in [kw]}$ and $\{(\mathbf{B}_i, \mathbf{v}_i)\}_{i \in [1, 2|\mathcal{U}|-2]}$ are sampled over $\mathbb{Z}_q^{n \times m} \times \mathbb{Z}_q^m$, thus $\Pr[\mu' = \mu \mid \mathcal{O} = \mathcal{O}'_{LWE}] = \frac{1}{2}$. Considering the successful execution of this IND-CKA game, the challenger $\mathcal{C}$ has advantage $\frac{\epsilon_1}{2}$ to solve the $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness.

Theorem 2. Assume that the $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness holds, our proposed lattice-based RABAEKS scheme satisfies IND-IKGA security in the ROM. For any PPT adversary $\mathcal{A}$, if $\mathcal{A}$ can compromise our scheme with a non-negligible advantage ϵ_2 , then a PPT challenger $\mathcal{C}$ can be constructed to solve the $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness with a non-negligible probability.

Proof 2. Assume that there exists a PPT adversary $\mathcal{A}$ that has the ability to break the IND-IKGA security with a non-negligible advantage ϵ_2 , we can construct a challenger $\mathcal{C}$ that solves the decisional $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness with the same advantage. In the Initialization phase, $\mathcal{A}$ sends a challenge attribute $(\mathcal{S}^*, \xi^*)$ and a revocation list $\mathcal{R}^*$ to $\mathcal{C}$, where $t^* \in [1, |\text{att}|]$. Let $\mathcal{ATT} = \{1, \dots, |\text{att}|\}$, where $\mathcal{W} \subseteq \mathcal{ATT}$, the interaction between $\mathcal{A}$ and $\mathcal{C}$ is showed as follows.

Setup: The challenger $\mathcal{C}$ samples $(\mathbf{A}_S, \mathbf{b})$ from the oracle $\mathcal{O}$, where $\mathcal{O} = \mathcal{O}_{LWE}$ or $\mathcal{O} = \mathcal{O}'_{LWE}$ meaning that this tuple is pseudo-random or random,

and invokes $(\mathbf{A}_w, \mathbf{T}_{\mathbf{A}_w}) \leftarrow \text{TrapGen}(n, m, q)$ to generate a matrix $\mathbf{A}_w \in \mathbb{Z}_q^{n \times m}$. Then, $\mathcal{C}$ selects many random matrices $\mathbf{A}_0 \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\mathbf{A}_a \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\overline{\mathbf{A}} \xleftarrow{\$} \mathbb{Z}_q^{n \times n}$, $\{\mathbf{A}_i\}_{i \in \mathcal{ATT}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\{\mathbf{U}_i\}_{i \in [1, |\mathcal{U}|-2]} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\{\mathbf{V}_i\}_{i \in [|\mathcal{U}|-1, 2|\mathcal{U}|-2]} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, and computes many matrices $\{\mathbf{W}_i = \mathbf{A}_S \mathbf{R}_i^* + \theta_i \mathbf{A}_w\}_{i \in [|\mathcal{K}|]}$ where $\boldsymbol{\theta} = (\theta_1, \dots, \theta_{|\mathcal{K}|}) \in \mathbb{Z}_q^{|\mathcal{K}|}$ and $\mathbf{R}_i^* \in \{-1, 1\}^{m \times m}$. After that, $\mathcal{C}$ sets a abort-resistant hash function $H : \mathbb{Z}_q^{|\mathcal{K}|} \setminus \{0\} \rightarrow \mathbb{Z}_q$ as defined in Definition 6. Finally, $\mathcal{C}$ sends the public parameter $pp = (n, m, q, \sigma, |\text{att}|, l, |\mathcal{K}|, \mathcal{ATT}, \mathcal{U}, \mathcal{R}, \mathbf{A}_0, \mathbf{A}_a, \mathbf{A}_w, \overline{\mathbf{A}}, \{\mathbf{A}_i\}_{i \in \mathcal{ATT}}, \{\mathbf{W}_i\}_{i \in [|\mathcal{K}|]}, \mathbf{B}_0, \{\mathbf{U}_i\}_{i \in [1, |\mathcal{U}|-2]}, \{\mathbf{V}_i\}_{i \in [|\mathcal{U}|-1, 2|\mathcal{U}|-2]}, \mathcal{T})$ to $\mathcal{A}$.

Phase 1: The adversary $\mathcal{A}$ executes these following queries adaptively.

- **Secret Key Queries** $\mathcal{O}_{RK}$: The query process is same as $\mathcal{O}_{RK}$ described in Theorem 1.
- **Ciphertext Queries** $\mathcal{O}_{CT}$: The adversary $\mathcal{A}$ queries the $\mathcal{O}_{CT}$ q_H times. For the each query, $\mathcal{A}$ inputs $((\mathcal{W}, t), \mathbf{ck})$, if $H(\mathbf{ck}) = 0$, this game is aborted, and $\mathcal{A}$ outputs $\mu' \in \{0, 1\}$ randomly. Otherwise, $\mathcal{C}$ calculates $\mathbf{c}_0$, $\mathbf{c}_1$, $\{\mathbf{c}_i\}_{i \in \mathcal{W}}$ and $\{\hat{\mathbf{c}}_k\}_{k \in \text{KUNodes}(\mathcal{T}, \mathcal{R}^*)}$ as same as in **Encrypt** $(pp, pk_S, sk_S, (\mathcal{W}, t), \mathbf{ck})$ algorithm. Then, $\mathcal{C}$ invokes $\mathbf{c}_2 \leftarrow \text{SampleRight}(\mathbf{A}_S, (1 + \sum_{i=1}^{|\mathcal{K}|} \theta_i \mathbf{c}_i) \mathbf{A}_w, \hat{\mathbf{R}}, \mathbf{T}_{\mathbf{A}_w}, \bar{\mathbf{c}}, \sigma)$ to generate a vector $\mathbf{c}_2 \in \mathbb{Z}_q^{2m}$, s.t. $(\mathbf{A}_S \mid (\mathbf{A}_w + \sum_{i=1}^{|\mathcal{K}|} \theta_i \mathbf{W}_i)) \mathbf{c}_2 = \bar{\mathbf{c}} \bmod q$, where $\hat{\mathbf{R}} = \sum_{i=1}^{|\mathcal{K}|} \theta_i \mathbf{R}_i^*$. Finally, $\mathcal{C}$ returns the ciphertext $CT := (\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \{\mathbf{c}_i\}_{i \in \mathcal{W}}, \{\hat{\mathbf{c}}_k\}_{k \in \text{KUNodes}(\mathcal{T}, \mathcal{R})})$ to $\mathcal{A}$.
- **Trapdoor Queries** $\mathcal{O}_{TD}$: After $\mathcal{A}$ inputs $((\mathcal{S}, \xi), \mathbf{tk})$, $\mathcal{C}$ executes **Trapdoor** $(pp, pk_S, sk_R, (\mathcal{S}, \xi), \mathbf{tk})$ to generate the ciphertext TD with the keyword $\mathbf{tk}$, and then returns it to $\mathcal{A}$.

Challenge: $\mathcal{A}$ selects two keywords $\mathbf{tk}_0^* := (tk_{0,1}^*, \dots, tk_{0,|\mathcal{K}|}^*)$ and $\mathbf{tk}_1^* := (tk_{1,1}^*, \dots, tk_{1,|\mathcal{K}|}^*)$ which have not been queried in **Phase 1**, and sends them to $\mathcal{C}$. Then, $\mathcal{C}$ selects a bit $\mu \in \{0, 1\}$ randomly. If $H(\mathbf{tk}_\mu^*) = 0$, this game is aborted, and $\mathcal{A}$ outputs $\mu' \in \{0, 1\}$ randomly. Otherwise, $\mathcal{C}$ generates the trapdoor TD_μ^* with the keyword $\mathbf{tk}_\mu^*$ and the attribute $(\mathcal{S}^*, \xi^*)$. Specially, $\mathcal{C}$ calculates $\mathbf{t}_0^* = \begin{pmatrix} \mathbf{b} \\ (\mathbf{R}^*)^\top \mathbf{b} \end{pmatrix}$, where $\mathbf{R}^* = \sum_{i=1}^{|\mathcal{K}|} tk_{\mu,i}^* \mathbf{R}_i^*$, and samples a random vector $\mathbf{t}_i^* \xleftarrow{\$} \mathbb{Z}_q^{4m}$ for $i \in \mathcal{S}^*$. Finally, $\mathcal{C}$ returns $\text{TD}_\mu^* = (\mathbf{t}_0^*, \{\mathbf{t}_i^*\}_{i \in \mathcal{S}^*})$ to $\mathcal{A}$.

Phase 2: The adversary $\mathcal{A}$ continues to query from $\mathcal{O}_{RK}$, $\mathcal{O}_{CT}$, and $\mathcal{O}_{TD}$ as same as in **Phase 1** except that the $((\mathcal{W}, t), \mathbf{ck})$ could be queried in $\mathcal{O}_{CT}$ when $\mathbf{ck} \notin (\mathbf{tk}_0^*, \mathbf{tk}_1^*)$.

Guess: The adversary $\mathcal{A}$ outputs $\mu' \in \{0, 1\}$ as the answer to $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness. Otherwise, $\mathcal{A}$ outputs $\mu' \in \{0, 1\}$ randomly. Then, the challenger $\mathcal{C}$ checks whether $H(\mathbf{tk}_\mu^*) = 0$ and $H(\mathbf{ck}_i) \neq 0$ for $i \in [q_H]$. If not, $\mathcal{C}$ overwrites $\mu' \in \{0, 1\}$ and aborts this game. After that, $\mathcal{C}$ selects a bit $\rho \in \{0, 1\}$, where $\Pr[\rho = 1] = \gamma(\mathbf{tk}_\mu^*, \mathbf{ck}_1, \dots, \mathbf{ck}_{q_H})$ and the function $\gamma()$ is defined in [54]. If $\rho = 1$, $\mathcal{C}$ overwrites $\mu' \in \{0, 1\}$ and aborts this game as an artificial abort. If $\mu' = \mu$, $\mathcal{A}$ wins this game and $\mathcal{C}$ outputs $\mathcal{O} = \mathcal{O}_{LWE}$. Otherwise, $\mathcal{C}$ outputs $\mathcal{O} = \mathcal{O}'_{LWE}$.

Analysis: If $\mu' = \mu$, $\mathcal{C}$ outputs $\mathcal{O} = \mathcal{O}_{LWE}$ meaning that $(\mathbf{A}_S, \mathbf{b})$ is sampled from $\mathcal{O}_{LWE}$ such that $\mathbf{b} = \mathbf{A}_S^\top \mathbf{s} + \mathbf{x}'_0$, where $\mathbf{s} \in \mathbb{Z}_q^n$ is a secret vector and $\mathbf{x}'_0 \leftarrow \Psi_\alpha^m$ is a noise vector. Then, due to $H(\mathbf{tk}_\mu^*) = 1 + \sum_{i=1}^{|kw|} tk_{\mu,i}^* \theta_i = 0$, we can obtain the following equations:

$$\begin{aligned} \mathbf{t}_0^* &= \begin{pmatrix} \mathbf{b} \\ (\mathbf{R}^*)^\top \mathbf{b} \end{pmatrix} = \begin{pmatrix} \mathbf{A}_S^\top \mathbf{s} + \mathbf{x}'_0 \\ (\mathbf{A}_S \mathbf{R}^*)^\top \mathbf{s} + (\mathbf{R}^*)^\top \mathbf{x}'_0 \end{pmatrix} \\ &= (\mathbf{A}_S \mid (\mathbf{A}_w + \sum_{i=1}^{|kw|} tk_{\mu,i}^* \mathbf{W}_i))^\top \mathbf{s} + \begin{pmatrix} \mathbf{x}'_0 \\ (\mathbf{R}^*)^\top \mathbf{x}'_0 \end{pmatrix} \\ \mathbf{t}_j^* &\leftarrow \mathbb{Z}_q^{4m}, j \in \mathcal{S} \end{aligned}$$

Consequently, $\text{TD}_\mu^* = (\mathbf{t}_0^*, \{\mathbf{t}_j\}_{j \in \mathcal{W}})$ is a valid trapdoor. From the Lemma 7 and $q_H \leq \frac{q}{2}$, we can obtain $|p_{\max} - p_{\min}| = \frac{1}{q} - \frac{1}{q}(1 - \frac{q_H}{q}) = \frac{q_H}{q^2}$, and $p_{\min} = \frac{1}{q}(1 - \frac{q_H}{q}) \geq \frac{1}{2q}$. To realize that $|p_{\max} - p_{\min}|$ is negligible, we need to make q large enough. After that, according to the Lemma 28 in [52], the advantage to solve the $\text{LWE}_{n,m,q,\Psi_\alpha}$ hardness is $\epsilon' \geq \frac{p_{\min}}{2} \epsilon_2 \geq \frac{\epsilon_2}{4q}$, considering the artificial abort of this IND-IKGA game.

7. Performance Evaluation and Comparison with Prior Arts

To ensure fairness in the performance evaluation, we selected several recent and high-quality lattice-based encrypted search schemes, including PAEKS [33], FS-IBEKS [38], BIB-MKS [39], and ABAEKS [20], as they are the most relevant to RABAEKS. Table 2 clearly highlights the functionality advantages of our proposed scheme.

Table 2: Functionality comparison with prior arts

Schemes	Encrypted Search	IKGAs-resilience	Attribute-based Solution	Data Receiver Revocation	Quantum Resistance
PAEKS [33]	✓	✓	✗	✗	✓
FS-IBEKS [38]	✓	✗	✗	✗	✓
BIB-MKS [39]	✓	✗	✗	✗	✓
ABAEKS [20]	✓	✓	✓	✗	✓
Our RABAEKS	✓	✓	✓	✓	✓

Table 3: Glossary in performance evaluation

Acronym	Definition
l	the security level of search
l_s	the execution number of SampleLeft for sender
l_r	the execution number of SampleLeft for receiver
$ \mathcal{BID} $	the number of possible values for each attribute
$ kw $	the number of keyword
$ \mathcal{W} $	the element number of access policy
$ \mathcal{S} $	the element number of receiver's attribute
t	the element number of subset $\mathcal{J} = \mathcal{S} \cap \mathcal{W}$
$ \mathcal{KU} $	the element number of $\text{KUNodes}(\mathcal{T}, \mathcal{R})$
T_{HASH}	the time of hash
T_{MUL}	the time of element multiplication over $\mathbb{Z}_q$
T_{INV}	the time of matrix inversion
T_{EPK}	the time of Eval_{pk}
T_{ECT}	the time of Eval_{ct}
T_{NBD}	the time of NewBasisDel
T_{SP}	the time of SamplePre
T_{SL}	the time of SampleLeft
T_{GSP}	the time of GenSamplePre
$ \mathbb{Z}_q $	the element length in $\mathbb{Z}_q$

The hardware environment consists of a laptop equipped with a 12th Gen Intel(R) Core(TM) i7-12800HX CPU, 16 GB RAM, and running the Windows 11 operating system. We utilize the Python language with the Numpy library to implement our RABAEKS and other schemes. Table 3 presents the symbols and definitions used in the performance evaluation. Based on the parameter settings, we select $q = 4097$, $n = 16$, $|\mathcal{U}| = 8$, $|\mathcal{KU}| = 2, 7, 10$, $l = 10$, $l_s = 4$, $l_r = 4$, and $|\mathcal{BID}| = 1$ to simulate the **Encrypt**, **Trapdoor**, and **Search** algorithms, assuming that the number of $\mathcal{S}$ is equal to the number of $\mathcal{W}$.

Table 4: Theoretical computational overhead

Schemes	Ciphertext generation	Trapdoor generation	Search phase
PAEKS [33]	$ kw * [T_{HASH} + (n^2m + n^2l_s + 3m^2l_s + 4nml_s)T_{MUL} + l_sT_{SL}]$	$ kw * [T_{HASH} + (n^2m + n^2l_r + 3m^2l_r + 4nml_r)T_{MUL} + l_rT_{SL}]$	$ kw * [8ml_sl_rT_{MUL}]$
FS-IBEKS [38]	$ kw * [T_{HASH} + T_{INV} + (nm^2 + nml + nl + l)T_{MUL}]$	$ kw * [T_{HASH} + nm^2T_{MUL} + T_{NBD} + T_{SP}]$	$ kw * [mlT_{MUL}]$
BIB-MKS [39]	$(\mathcal{BID} + kw + 1)T_{HASH} + (\mathcal{BID} + 1)T_{INV} + [\mathcal{BID} (2nm^2 + (nm + m)l + (n + 2)l)]T_{MUL}$	$(kw + 1)T_{HASH} + 2 \mathcal{BID} T_{INV} + 2 \mathcal{BID} nm^2T_{MUL} + \mathcal{BID} T_{NBD} + \mathcal{BID} T_{SP}$	$2tmlT_{MUL}$
ABAEKS [20]	$ kw * [T_{HASH} + [(2 \mathcal{S} + 3)nm + n^2 + (\mathcal{S} + 1)m^2]T_{MUL} + T_{SL}]$	$ kw * [T_{HASH} + T_{EPK} + (n^2 + m^2 + 3nm)T_{MUL} + T_{SL}]$	$ kw * [T_{ECT} + 5mT_{MUL}]$
Our RABAEKS	$[(kw + \mathcal{W} + \mathcal{KU} + 2)nm + n^2]T_{MUL} + T_{SL}$	$[(kw + 2)nm + (kw + 1)m^2 + n^2]T_{MUL} + \mathcal{S} T_{GSP}$	$(4tm + 2m)T_{MUL}$

7.1. Computational overhead

From a theoretical perspective, we analyze the computational overhead of the five schemes presented in Table 4. It is evident that the execution time for ciphertext generation, trapdoor generation, and the search phase in PAEKS, FS-IBEKS, and ABAEKS increases with the number of keywords $|kw|$. Additionally, the search time in BIB-MKS and RABAEKS remains independent of $|kw|$.

Fig. 4 illustrates the computational overhead comparison between our RABAEKS scheme and prior works in ciphertext generation, trapdoor generation, and the search phase, as the number of keywords $|kw|$ increases. In Fig. 4(a), the execution time for PAEKS, FS-IBEKS, and ABAEKS is proportional to $|kw|$ during the ciphertext calculation algorithm. BIB-MKS and RABAEKS can encrypt multiple keywords simultaneously, and are more efficient compared to the other three schemes, with RABAEKS exhibiting significantly lower computational overhead than BIB-MKS. Regarding Fig. 4(b), both ABAEKS and our RABAEKS scheme avoid matrix inversion operations and the **NewBasisDel** algorithm, resulting in lower trapdoor generation times compared to PAEKS, FS-IBEKS, and BIB-MKS. The actual results show that trapdoor generation in RABAEKS is slightly more efficient than in ABAEKS. For the search phase, as shown in Fig. 4(c), the computational overhead of PAEKS, FS-IBEKS, and ABAEKS increases linearly with the number of keywords $|kw|$. In contrast, BIB-MKS and RABAEKS simultaneously hash-map multiple keywords into vectors of the same dimension, so the number of keywords does not affect search time. When the cloud server handles a large number of keyword ciphertexts and search requests, our solution proves to be significantly more practical due to its enhanced search

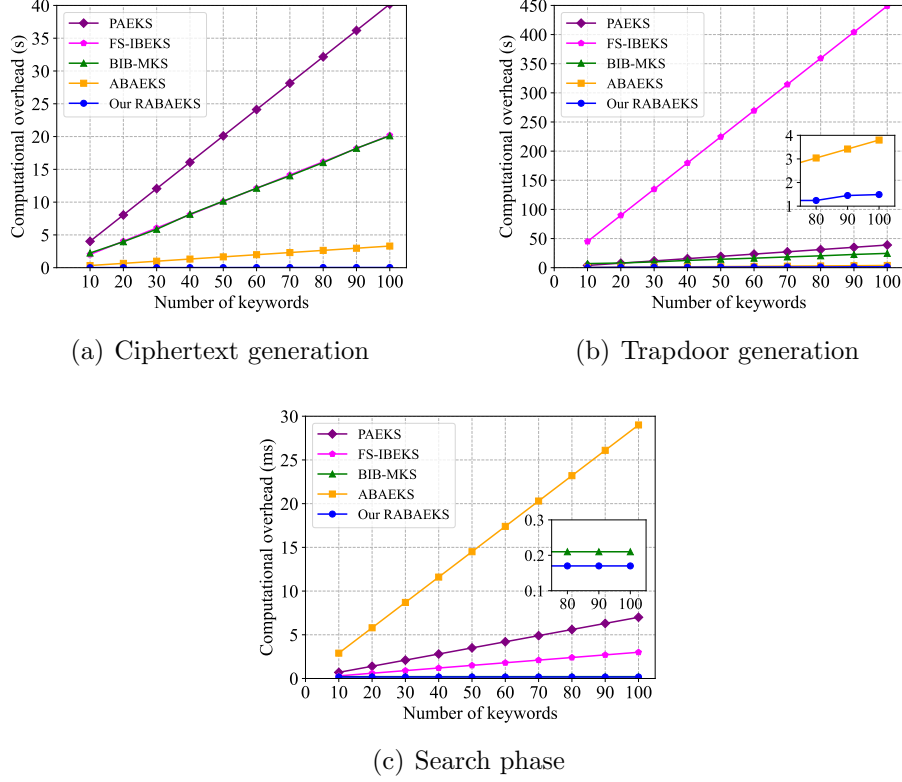


Figure 4: Computational overhead comparison between our RABAEKS and other prior arts with the number of keywords $|kw|$.

efficiency.

Since our RABAEKS does not involve the time-consuming matrix inverse and **NewBasisDel** algorithms utilized in BIB-MKS, a single execution can handle multiple keywords simultaneously, without requiring repeated invocations as in PAEKS, FS-IBEKS, and ABAEKS. Therefore, when the number of keywords $|kw| = 100$ as shown in Table 5, the execution times for ciphertext generation, trapdoor generation, and the search phase are 0.03s, 1.49s, and 0.17ms, respectively, corresponding to only 0.91%, 39.21%, and 80.95% of those in prior works. These results demonstrate that our RABAEKS is highly efficient in processing multiple keywords, and this advantage becomes even more significant as the keyword set size increases in cloud network environments.

As depicted in Fig. 5, we evaluate the effects of varying numbers of re-

Table 5: Numerical computational overhead comparison when $|kw| = 100$

Schemes	Ciphertext generation	Trapdoor generation	Search phase
PAEKS [33]	40.24s	38.89s	7.00ms
FS-IBEKS [38]	20.25s	449.08s	3.00ms
BIB-MKS [39]	20.11s	24.58s	0.21ms
ABAEKS [20]	3.28s	3.80s	29.00ms
Our RABAEKS	0.03s	1.49s	0.17ms

Table 6: Theoretical communication overhead

Schemes	Secret key of receiver	Ciphertext	Trapdoor
PAEKS [33]	$m^2 \mathbb{Z}_q $	$8ml_s \mathbb{Z}_q $	$8ml_r \mathbb{Z}_q $
FS-IBEKS [38]	$m^2 \mathbb{Z}_q $	$l(m+1) \mathbb{Z}_q $	$m \mathbb{Z}_q $
BIB-MKS [39]	$ \mathcal{BTD} m^2 \mathbb{Z}_q $	$(\mathcal{BTD} lm + \mathcal{BTD} n + l) \mathbb{Z}_q $	$ \mathcal{BTD} (n+m) \mathbb{Z}_q $
ABAEKS [20]	$4m^2 \mathbb{Z}_q $	$(\mathcal{S} + 4)m \mathbb{Z}_q $	$5m \mathbb{Z}_q $
Our RABAEKS	$4m^2 \mathcal{S} \mathbb{Z}_q $	$(\mathcal{W} + \mathcal{KU} + 4)m \mathbb{Z}_q $	$(4 \mathcal{S} + 2)m \mathbb{Z}_q $

ceiver attributes (i.e., $|\mathcal{S}| = 10, 20, 30, 40, 50$) on ciphertext generation, trapdoor generation, and the search phase in ABAEKS and RABAEKS. Notably, since PAEKS, FS-IBEKS, and BIB-MKS do not involve receiver attributes, these three schemes are not included in Fig. 5. In Fig. 5(a), the ciphertext generation algorithm in RABAEKS is more efficient and less influenced by the number of receiver attributes $|\mathcal{S}|$, compared to ABAEKS. In Fig. 5(b), the computational overheads of ABAEKS and RABAEKS are similar when $|\mathcal{S}| = 10$. However, since the number of calls to the `GenSamplePre` algorithm in RABAEKS is entirely dependent on $|\mathcal{S}|$, the time of our trapdoor generation algorithm is more significantly affected by $|\mathcal{S}|$, resulting in lower efficiency compared to ABAEKS. Additionally, as shown in Fig. 5(c), the computational overhead of RABAEKS in the search phase is considerably lower than that of ABAEKS, similar to the ciphertext generation algorithm.

7.2. Communication overhead

For searchable encryption schemes, the communication overhead depends on the size of the receiver's secret key, the ciphertext size, and the trapdoor size. On the one hand, the theoretical communication overhead comparison between our RABAEKS scheme and other schemes [33], [38], [39], [20] is presented in Table 6.

On the other hand, we analyze the communication overhead of the receiver's secret key in RABAEKS compared to other prior arts. In RABAEKS,

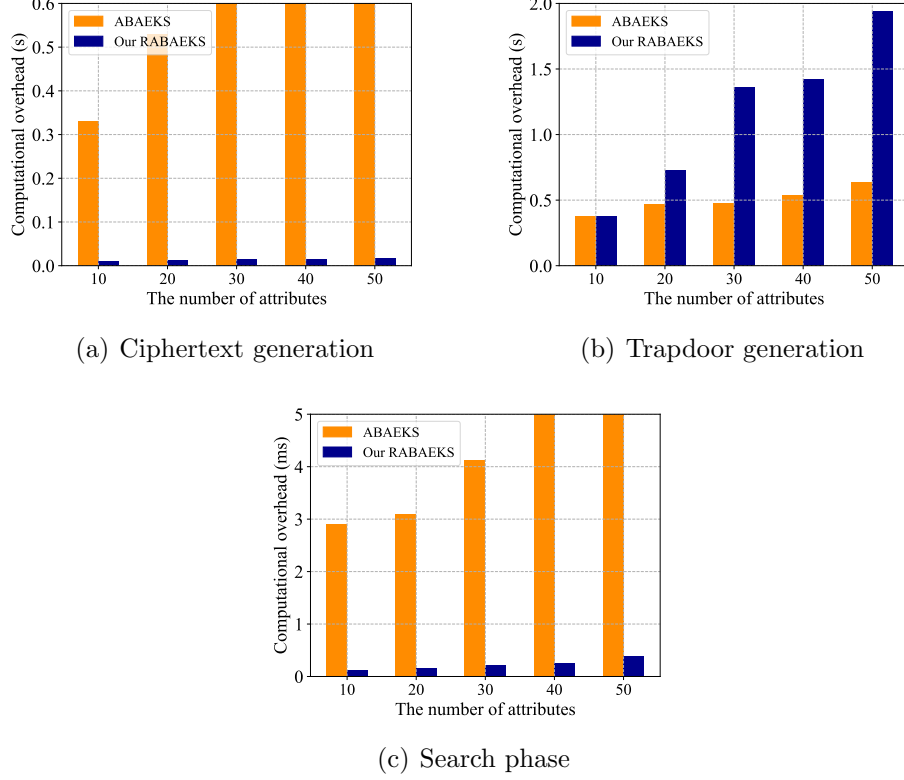


Figure 5: Computational overhead comparison between our RABAEKS and ABAEKS [20] with the number of receiver’s attributes $|\mathcal{S}|$. Since PAEKS [33], FS-IBEKS [38], and BIB-MKS [39] do not involve attribute numbers, these three schemes are not included.

the secret key is embedded in the data receiver’s attribute matrix, and its size is proportional to the number of attributes $|\mathcal{S}|$. Therefore, it does not offer any advantage over the other four schemes. As depicted in Fig. 6(b) and 6(c), we analyze the communication overhead in terms of ciphertext and trapdoor size as the number of keywords $|kw|$ increases. For PAEKS, FS-IBEKS, and ABAEKS, both ciphertext and trapdoor sizes increase linearly with $|kw|$, while BIB-MKS and RABAEKS maintain constant ciphertext and trapdoor sizes. For instance, with parameter settings $|\mathbb{Z}_q| = 13$, $n = 16$, $m = \lceil 2n \log q \rceil = 385$, $|kw| = 100$, $|\mathcal{S}| = |\mathcal{W}| = 10$, and $|\mathcal{KU}| = 3$, the secret key size is $4m^2|\mathcal{S}||\mathbb{Z}_q| = 4 \times 385^2 \times 10 \times 13 = 9.19MB$, the ciphertext size is $(|\mathcal{W}| + |\mathcal{KU}| + 4)m|\mathbb{Z}_q| = (10 + 3 + 4) \times 385 \times 13 = 10.39KB$, and the trapdoor size is $(4|\mathcal{S}| + 2)m|\mathbb{Z}_q| = (4 \times 10 + 2) \times 385 \times 13 = 25.66KB$. The

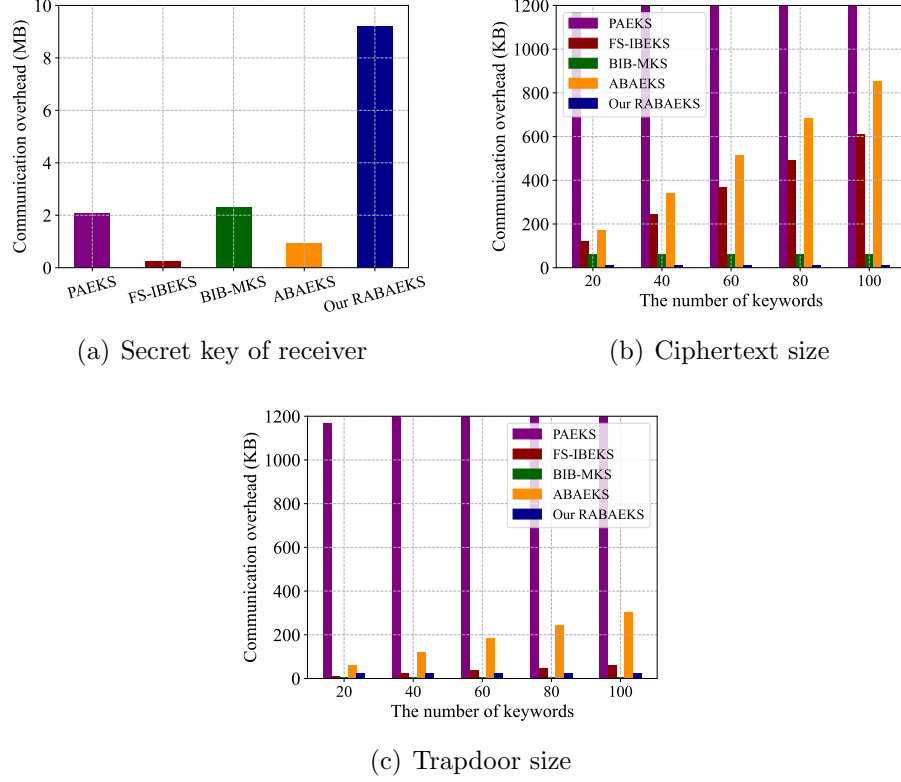


Figure 6: Communication overhead comparison between our RABAEKS and other prior arts.

comparison results show that RABAEKS has a relatively low communication overhead in terms of ciphertext and trapdoor sizes. Although the secret key size in RABAEKS is large, it provides both access control and attribute revocation in cloud networks, making it more practical compared to the other four schemes.

8. Conclusion and Future Work

In this paper, we propose RABAEKS, a revocable and authenticated attribute-based encrypted search scheme over lattices in cloud networks. Our design incorporates attributes into the data receiver’s secret key and access policies into the keyword ciphertext to enable multi-receiver access control. Additionally, we introduce a receiver revocation mechanism, thereby con-

structuring a more dynamic access control framework. The rigorous security analysis demonstrates that RABAEKS achieves IND-CKA and IND-IKGA security in the ROM. The comprehensive experimental evaluations also indicate that RABAEKS is more efficient than other prior art in terms of ciphertext generation, trapdoor generation, and search algorithms.

Although our design reduces IND-CKA and IND-IKGA security to the LWE assumption, it has only been proven in the ROM. Therefore, enhancing security in the standard model presents an interesting direction for future research. Besides, our scheme is not resistant to secret key leakage attacks, meaning that keyword security cannot be guaranteed if the data receiver's secret key is compromised. Addressing this vulnerability represents another valuable area for exploration.

Acknowledgement

This work was supported by NSFC (Grant Nos. 72293583, 72293580); BUPT Excellent Ph.D. Students Foundation (Grant No. CX20241060); and the Open Foundation of State Key Laboratory of Networking and Switching Technology (Beijing University of Posts and Telecommunications) under Grant SKLNST-2024-1-06.

References

- [1] Y. Zhao, J. Chang, Certificateless public auditing scheme with designated verifier and privacy-preserving property in cloud storage, *Computer Networks* 216 (2022) 109270.
- [2] Z. Liu, S. Wang, Y. Liu, Blockchain-based integrity auditing for shared data in cloud storage with file prediction, *Computer Networks* 236 (2023) 110040.
- [3] C. Huang, D. Liu, A. Yang, R. Lu, X. Shen, Multi-client secure and efficient dpf-based keyword search for cloud storage, *IEEE Transactions on Dependable and Secure Computing* 21 (1) (2023) 353–371.
- [4] J. Sun, W. Su, L. Wang, K. Ye, Multi-keyword ranked search scheme for privacy protection in social networks, *Computer Networks* 249 (2024) 110534.

- [5] G. Xu, D.-L. Kong, K. Zhangs, S. Xu, Y. Cao, Y. Mao, J. Duan, J. Kang, X.-B. Chen, A model value transfer incentive mechanism for federated learning with smart contracts in aiot, *IEEE Internet of Things Journal* 12 (3) (2025) 2530–2544. doi:10.1109/JIOT.2024.3468443.
- [6] P. Duan, H. Gao, Y. Shen, Z. Guo, Z. Ma, T. Tian, Y. Zhang, Secure collaborative ehr sharing using multi-authority attribute-based proxy re-encryption in web 3.0, *Computer Networks* 255 (2024) 110851.
- [7] W. Luo, Z. Lv, C. Lai, T. Yang, Efficient and secure cross-domain data sharing scheme with traceability for industrial internet, *Computer Networks* (2025) 111117.
- [8] Y. Cao, S. Xu, X. Chen, Y. He, S. Jiang, A forward-secure and efficient authentication protocol through lattice-based group signature in vanets scenarios, *Computer Networks* 214 (2022) 109149.
- [9] G. Xu, S. Xu, X. Fan, Y. Cao, Y. Mao, Y. Xie, X.-B. Chen, Rat ring: Event driven publish/subscribe communication protocol for iiot by report and traceable ring signature, *IEEE Transactions on Industrial Informatics* (2025) 1–9doi:10.1109/TII.2025.3567265.
- [10] G. Xu, X. Fan, S. Xu, Y. Cao, X.-B. Chen, T. Shang, S. Yu, Anonymity-enhanced sequential multi-signer ring signature for secure medical data sharing in iomt, *IEEE Transactions on Information Forensics and Security* (2025) 1–1doi:10.1109/TIFS.2025.3574959.
- [11] Q. Huang, H. Li, An efficient public-key searchable encryption scheme secure against inside keyword guessing attacks, *Information Sciences* 403 (2017) 1–14.
- [12] H. Li, Q. Huang, J. Huang, W. Susilo, Public-key authenticated encryption with keyword search supporting constant trapdoor generation and fast search, *IEEE Transactions on Information Forensics and Security* 18 (2023) 396–410. doi:10.1109/TIFS.2022.3224308.
- [13] L. Cheng, F. Meng, Server-aided public key authenticated searchable encryption with constant ciphertext and constant trapdoor, *IEEE Transactions on Information Forensics and Security* 19 (2024) 1388–1400. doi:10.1109/TIFS.2023.3336160.

- [14] A. Wu, F. Li, X. Xin, Y. Zhang, J. Zhu, Efficient public-key searchable encryption against inside keyword guessing attacks for cloud storage, *Journal of Systems Architecture* 149 (2024) 103104.
- [15] G. Xu, S. Xu, Y. Cao, K. Xiao, Y. Mao, X.-B. Chen, M. Dong, S. Yu, Aaq-peks: An attribute-based anti-quantum public key encryption scheme with keyword search for e-healthcare scenarios, *Peer-to-Peer Networking and Applications* 18 (2) (2025) 1–21.
- [16] F. Luo, X. Yan, H. Yang, X. Zheng, Paews: Public-key authenticated encryption with wildcard search over outsourced encrypted data, *IEEE Transactions on Information Forensics and Security* 20 (2025) 2212–2223. doi:10.1109/TIFS.2025.3540606.
- [17] V. B. Chenam, S. T. Ali, A designated cloud server-based multi-user certificateless public key authenticated encryption with conjunctive keyword search against ikga, *Computer Standards & Interfaces* 81 (2022) 103603.
- [18] L. Sun, Z. Cao, X. Dong, J. Shen, M. Wang, J. Chen, dmclaeks: Pairing-free designated-tester multi-recipient certificateless authenticated encryption with keyword search for concealing search patterns, *Journal of Systems Architecture* 144 (2023) 103007.
- [19] Y. Xu, H. Cheng, J. Li, X. Liu, X. Zhang, M. Wang, Lightweight multi-user public-key authenticated encryption with keyword search, *IEEE Transactions on Information Forensics and Security* 20 (2025) 3234–3246. doi:10.1109/TIFS.2025.3550054.
- [20] F. Luo, H. Wang, C. Lin, X. Yan, Abaeks: Attribute-based authenticated encryption with keyword search over outsourced encrypted data, *IEEE Transactions on Information Forensics and Security* 18 (2023) 4970–4983. doi:10.1109/TIFS.2023.3301740.
- [21] X. Liu, Z. Wei, G. Li, J. Chen, An enhanced traceable access control scheme based on multi-authority cp-abe for cloud-assisted e-health system, *Computer Networks* 254 (2024) 110766.
- [22] Y. Yang, J. Zhang, Y. Chen, Q. Huang, F. Chen, J. Chen, An efficient encrypted search with owner-level and attribute-level access controls, *Computer Networks* 258 (2025) 111039.

- [23] Q. Huang, Q. Wei, G. Yan, L. Zou, Y. Yang, Fast and privacy-preserving attribute-based keyword search in cloud document services, *IEEE Transactions on Services Computing* 16 (5) (2023) 3348–3360. doi:10.1109/TSC.2023.3265270.
- [24] H. Yin, Y. Zhao, L. Zhang, B. Qiao, W. Chen, H. Wang, Attribute-based searchable encryption with decentralized key management for healthcare data sharing, *Journal of Systems Architecture* 148 (2024) 103081.
- [25] D. Ghopur, J. Ma, X. Ma, F. He, K. Liu, T. Jiang, X. Wang, Decentralized multiauthority attribute-based searchable encryption for e-health cloud, *IEEE Internet of Things Journal* 12 (11) (2025) 15723–15735. doi:10.1109/JIOT.2025.3529480.
- [26] R. Hou, F. Zhou, Q. Wang, Z. Jiao, J. Sun, Z. Zhang, Revokable blockchain-enabled ranked multi-keyword attribute-based searchable encryption scheme with mobile edge computing for vehicular, *IEEE Transactions on Network and Service Management* 22 (3) (2025) 2764–2779. doi:10.1109/TNSM.2025.3557262.
- [27] X. Chen, S. Xu, Y. Cao, Y. He, K. Xiao, Aqrs: Anti-quantum ring signature scheme for secure epidemic control with blockchain, *Computer Networks* 224 (2023) 109595.
- [28] B. Zhang, W. Yang, F. Zhang, J. Ning, Efficient attribute-based searchable encryption with policy hiding over personal health records, *IEEE Transactions on Dependable and Secure Computing* 22 (2) (2025) 1299–1312. doi:10.1109/TDSC.2024.3432769.
- [29] C. Ge, W. Susilo, J. Baek, Z. Liu, J. Xia, L. Fang, Revocable attribute-based encryption with data integrity in clouds, *IEEE Transactions on Dependable and Secure Computing* 19 (5) (2021) 2864–2872.
- [30] F. Luo, S. Al-Kuwari, H. Wang, F. Wang, K. Chen, Revocable attribute-based encryption from standard lattices, *Computer Standards & Interfaces* 84 (2023) 103698.
- [31] L. Guo, L. Wang, X. Ma, Q. Ma, A new revocable attribute based encryption on lattice, in: *International Conference on Provable Security*, Springer, 2023, pp. 309–326.

- [32] X. Shen, X. Li, H. Yin, C. Cao, L. Zhang, Lattice-based multi-authority ciphertext-policy attribute-based searchable encryption with attribute revocation for cloud storage, *Computer Networks* 250 (2024) 110559.
- [33] L. Cheng, F. Meng, Public key authenticated encryption with keyword search from lwe, in: *European Symposium on Research in Computer Security*, Springer, 2022, pp. 303–324.
- [34] M. Yang, H. Wang, D. He, Pm-abe: Puncturable bilateral fine-grained access control from lattices for secret sharing, *IEEE Transactions on Dependable and Secure Computing* (2024).
- [35] P. W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, *SIAM review* 41 (2) (1999) 303–332.
- [36] S. Xu, Y. Cao, X. Chen, Y. Zhao, S.-M. Yiu, Post-quantum public-key authenticated searchable encryption with forward security: General construction, and applications, in: *International Conference on Information Security and Cryptology*, Springer, 2023, pp. 274–298.
- [37] S. Wang, C. Xu, G. Zhao, Z. Han, R. Hu, S. Yu, Apqa: An anonymous post quantum access authentication scheme based on lattice for space ground integrated network, *Computer Networks* 257 (2025) 110979.
- [38] X. Yang, X. Chen, J. Huang, H. Li, Q. Huang, Fs-ibeks: Forward secure identity-based encryption with keyword search from lattice, *Computer Standards & Interfaces* 86 (2023) 103732.
- [39] X. Zhang, C. Huang, D. Gu, J. Zhang, H. Wang, Bib-mks: Post-quantum secure biometric identity-based multi-keyword search over encrypted data in cloud storage systems, *IEEE Transactions on Services Computing* 16 (1) (2023) 122–133. doi:10.1109/TSC.2021.3112779.
- [40] S. Xu, Y. Cao, X. Chen, Y. Guo, Y. Yang, F. Guo, S.-M. Yiu, Post-quantum searchable encryption supporting user-authorization for outsourced data management, in: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2024, pp. 2702–2711.

- [41] A. Boldyreva, V. Goyal, V. Kumar, Identity-based encryption with efficient revocation, in: Proceedings of the 15th ACM conference on Computer and communications security, 2008, pp. 417–426.
- [42] Y. Miao, F. Li, X. Li, Z. Liu, J. Ning, H. Li, K.-K. R. Choo, R. H. Deng, Time-controllable keyword search scheme with efficient revocation in mobile e-health cloud, *IEEE Transactions on Mobile Computing* 23 (5) (2024) 3650–3665. doi:10.1109/TMC.2023.3277702.
- [43] F. Meng, L. Cheng, Str-abks: Server-aided traceable and revocable attribute-based encryption with keyword search, *IEEE Internet of Things Journal* 11 (7) (2024) 12649–12659. doi:10.1109/JIOT.2023.3335624.
- [44] Z.-Y. Liu, Y.-F. Tseng, R. Tso, M. Mambo, Y.-C. Chen, Public-key authenticated encryption with keyword search: A generic construction and its quantum-resistant instantiation, *The Computer Journal* 65 (10) (2022) 2828–2844.
- [45] F. Luo, H. Wang, X. Yan, Re-paeks: Public-key authenticated re-encryption with keyword search, *IEEE Transactions on Mobile Computing* 23 (10) (2024) 10077–10092. doi:10.1109/TMC.2024.3373626.
- [46] S. Zhao, R. Jiang, B. Bhargava, Rl-abe: A revocable lattice attribute based encryption scheme based on r-lwe problem in cloud storage, *IEEE Transactions on Services Computing* 15 (2) (2020) 1026–1035.
- [47] M. Ajtai, Generating hard instances of lattice problems, in: Proceedings of the twenty-eighth annual ACM symposium on Theory of computing, 1996, pp. 99–108.
- [48] C. Peikert, An efficient and parallel gaussian sampler for lattices, in: Annual Cryptology Conference, Springer, 2010, pp. 80–97.
- [49] O. Regev, On lattices, learning with errors, random linear codes, and cryptography, *Journal of the ACM (JACM)* 56 (6) (2009) 1–40.
- [50] C. Gentry, C. Peikert, V. Vaikuntanathan, Trapdoors for hard lattices and new cryptographic constructions, in: Proceedings of the fortieth annual ACM symposium on Theory of computing, 2008, pp. 197–206.

- [51] D. Micciancio, C. Peikert, Trapdoors for lattices: Simpler, tighter, faster, smaller., in: Eurocrypt, Vol. 7237, Springer, 2012, pp. 700–718.
- [52] S. Agrawal, D. Boneh, X. Boyen, Efficient lattice (h) ibe in the standard model, in: Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29, Springer, 2010, pp. 553–572.
- [53] D. Cash, D. Hofheinz, E. Kiltz, C. Peikert, Bonsai trees, or how to delegate a lattice basis, Journal of cryptology 25 (2012) 601–639.
- [54] B. Waters, Efficient identity-based encryption without random oracles, in: Advances in Cryptology–EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22–26, 2005. Proceedings 24, Springer, 2005, pp. 114–127.